

清華大學 交叉信息研究院
Tsinghua University Institute for Interdisciplinary Information Sciences


ISR Lab
Intelligent Systems and Robotics Lab

Advanced Topics for Robotics
智能机器人前沿探究

Lecture 6: Advanced Policy Learning

Jianyu Chen (陈建宇)

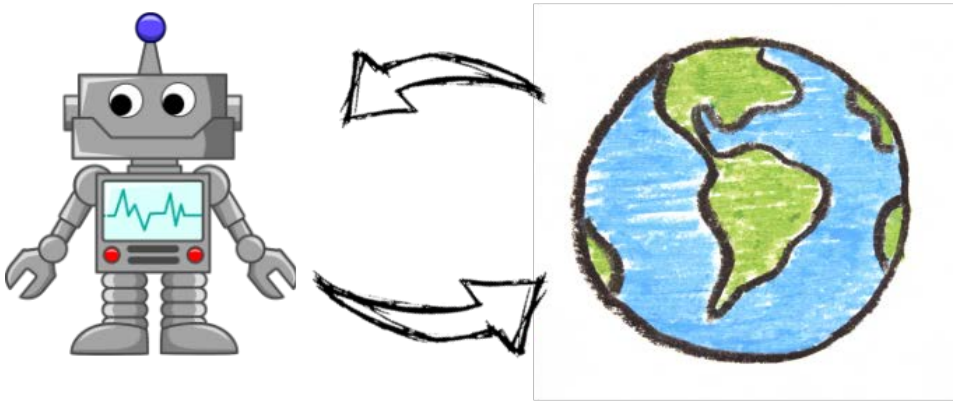
Institute for Interdisciplinary Information Sciences

Tsinghua University

Contents

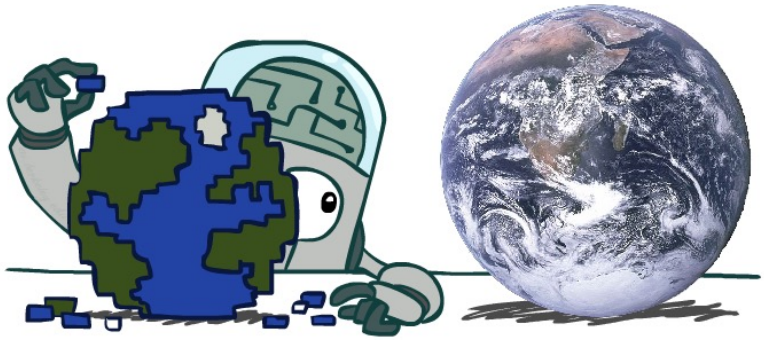
Model-free vs Model-based RL

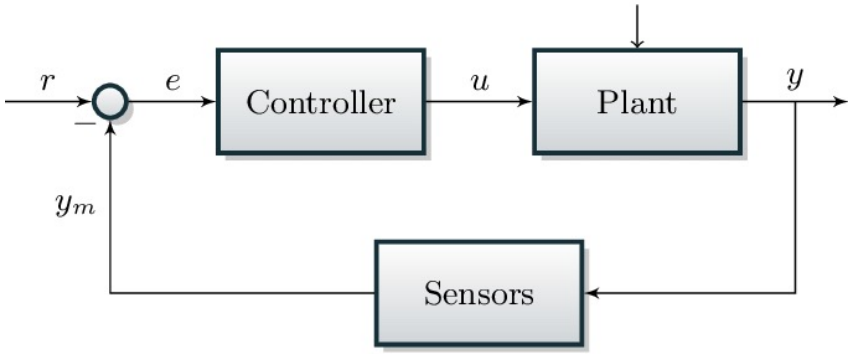
- Previous model-free RL work in a pure trial-and-error style
- This leads to low-sample efficiency



Model-free vs Model-based RL

- However, we human can learn to model the world, which helps us better adapt to new environments
- Control (Optimal control, MPC) also work in a model-based way





Model-Based Reinforcement Learning

- How to do RL in a model-based style?
  - Learn the model, then use the model to plan
  - Do policy gradient through the learned differentiable model
  - Use the learned model to generate data, then use model-free RL
  - Learn a latent space model for POMDP, then do latent space planning

Model-Based Reinforcement Learning

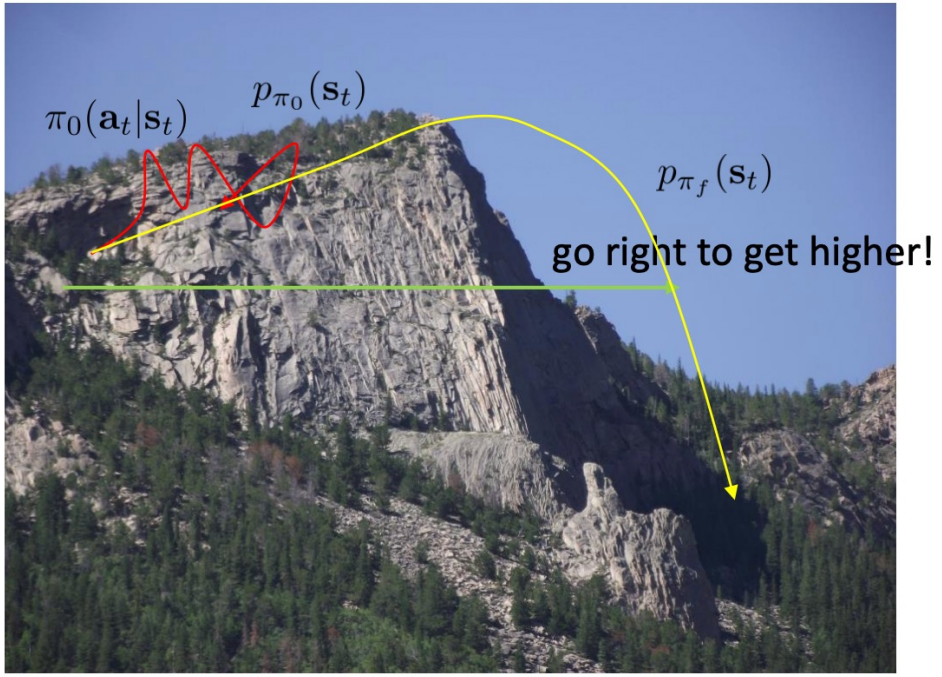
- Intuitively, we can learn the model first, and then perform control
- How to generate the data for model learning?
- Model-Based RL version 1:
  - Run based policy (e.g, a random policy) to collect data $\mathcal{D} = \{x_k, a_k\}$
  - Learn dynamics model, e.g.:

$$\min_w \sum_{\mathcal{D}} \| f_w(x_k, u_k) - x_{k+1} \|^2$$

  - Model-based control (e.g, MPC with trajectory optimization) with learned dynamics

Model-Based Reinforcement Learning

- Any problem of this?
- You think the dynamics is to go uphill all the way
- However you have not seen the data where there is a cliff



- Another example:
  - For a pendulum, if the base policy only provide data where the pendulum is down, you might not be able to obtain a controller for it to be upright

Model-Based Reinforcement Learning

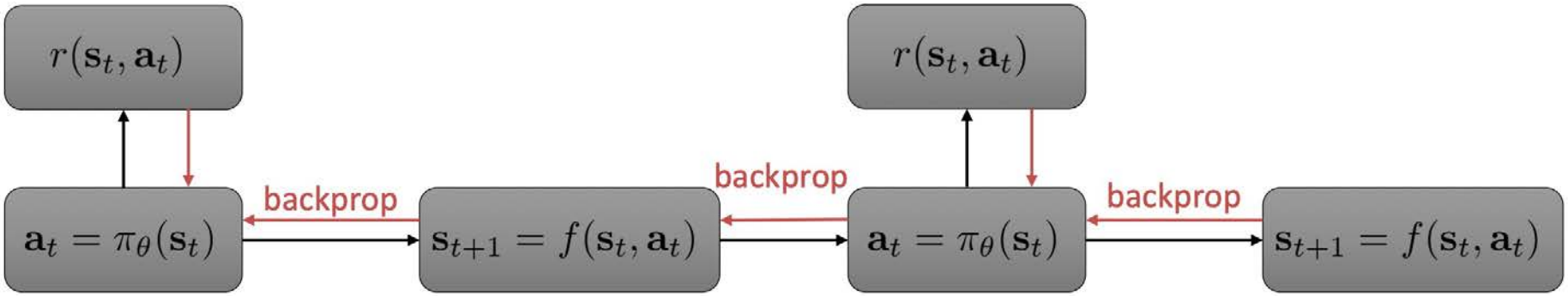
- Instead of learning the model offline, we can interactively learn the model and do control both online
- Model-Based RL version 2:
  - Run based policy (e.g, a random policy) to collect data $\mathcal{D} = \{x_k, a_k\}$
  - Learn dynamics model, e.g.:

$$\min_w \sum_{\mathcal{D}} \| f_w(x_k, u_k) - x_{k+1} \|^2$$

  - Model-based control (e.g, MPC with trajectory optimization) with learned dynamics
  - Collect new data and add to the dataset $\mathcal{D}$

Model-Based Reinforcement Learning

- With a differentiable model, we can also use backpropagation through time (BPTT) to obtain the corresponding model-based policy

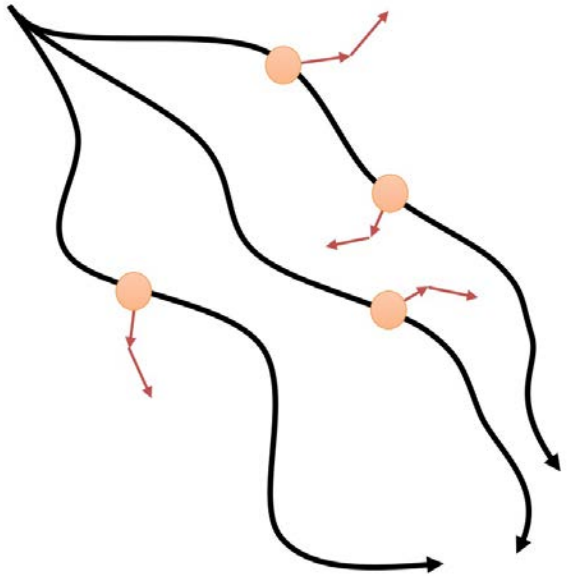


1. run base policy $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$ (e.g., random policy) to collect $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$
2. learn dynamics model $f(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$
3. backpropagate through $f(\mathbf{s}, \mathbf{a})$ into the policy to optimize $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$
4. run $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$, appending the visited tuples $(\mathbf{s}, \mathbf{a}, \mathbf{s}')$ to $\mathcal{D}$

Model-Based Reinforcement Learning

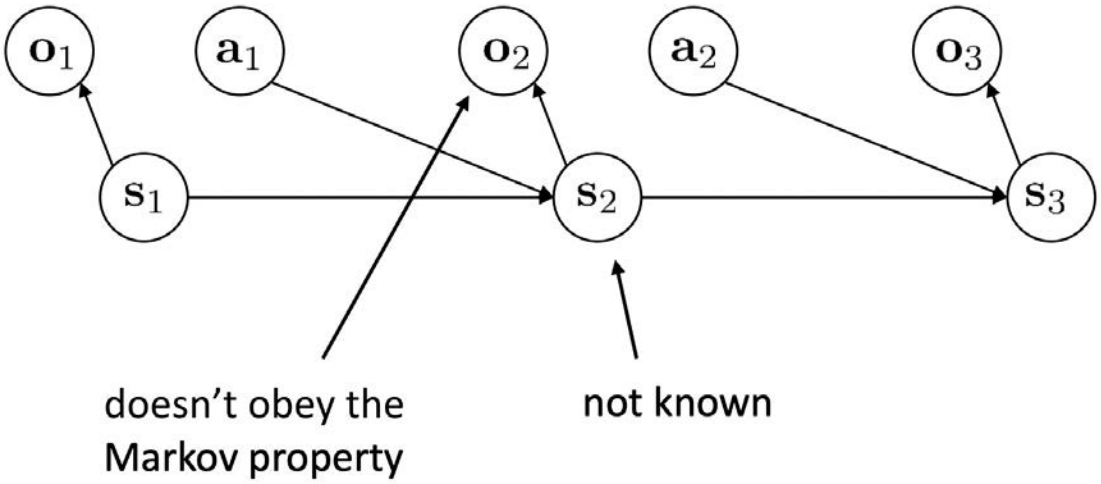
- Instead of use model to plan, we can also use model to augment the data, and then use model-free RL

1. collect some data, consisting of transitions (s, a, s', r)
2. learn model $\hat{p}(s'|s, a)$ (and optionally, $\hat{r}(s, a)$)
3. repeat K times:
   4. sample $s \sim \mathcal{B}$ from buffer
   5. choose action a (from $\mathcal{B}$, from π, or random)
   6. simulate $s' \sim \hat{p}(s'|s, a)$ (and $r = \hat{r}(s, a)$)
   7. train on (s, a, s', r) with model-free RL
   8. (optional) take N more model-based steps



Latent Space Model

- Real world applications are beyond MDP
- The partially observable issue

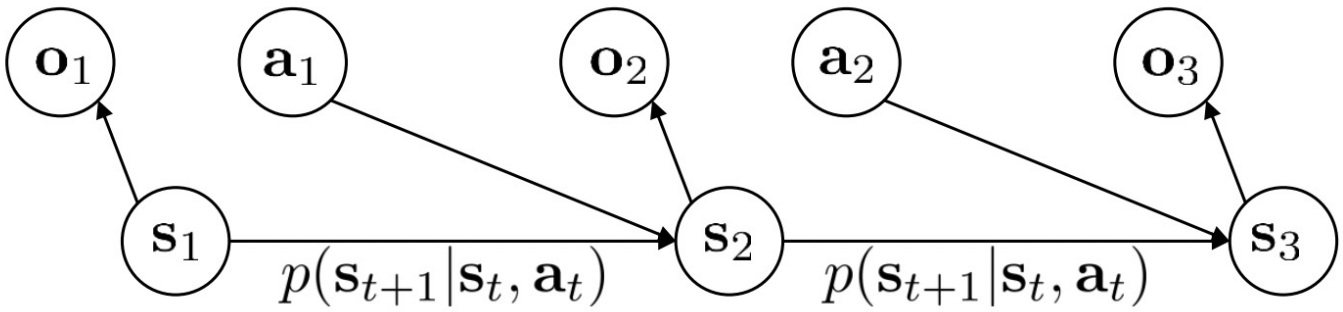


$\mathbf{o}_t$ – observation

$\mathbf{s}_t$ – state

Latent Space Model

- Now consider the general nonlinear, stochastic, and high-dimensional state space model, often called latent space model
- How to learn this latent space model, in an unsupervised way?



Learning Latent Space Model

- For dynamic model learning:

$$\min_{\phi} \sum_{\mathcal{D}} \| f_{\phi}(s_t, u_t) - s_{t+1} \|^2$$

$$\max_{\phi} \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \log p_{\phi}(\mathbf{s}_{t+1,i} | \mathbf{s}_{t,i}, \mathbf{a}_{t,i})$$

- For latent space model learning

$$\max_{\phi} \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} E\left[\log p_{\phi}(\mathbf{s}_{t+1,i} | \mathbf{s}_{t,i}, \mathbf{a}_{t,i}) + \log p_{\phi}(\mathbf{o}_{t,i} | \mathbf{s}_{t,i})\right]$$

$$(\mathbf{s}_t, \mathbf{s}_{t+1}) \sim p(\mathbf{s}_t, \mathbf{s}_{t+1} | \mathbf{o}_{1:T}, \mathbf{a}_{1:T})$$

Learning Latent Space Model

- Can approximate the expectation term

$$(\mathbf{s}_t, \mathbf{s}_{t+1}) \sim p(\mathbf{s}_t, \mathbf{s}_{t+1} | \mathbf{o}_{1:T}, \mathbf{a}_{1:T})$$

with $$\mathbf{s}_t \sim q_\psi(\mathbf{s}_t|\mathbf{o}_t), \mathbf{s}_{t+1} \sim q_\psi(\mathbf{s}_{t+1}|\mathbf{o}_{t+1})$$

- This requires variational inference
- A simplified version is to make q_ϕ deterministic:

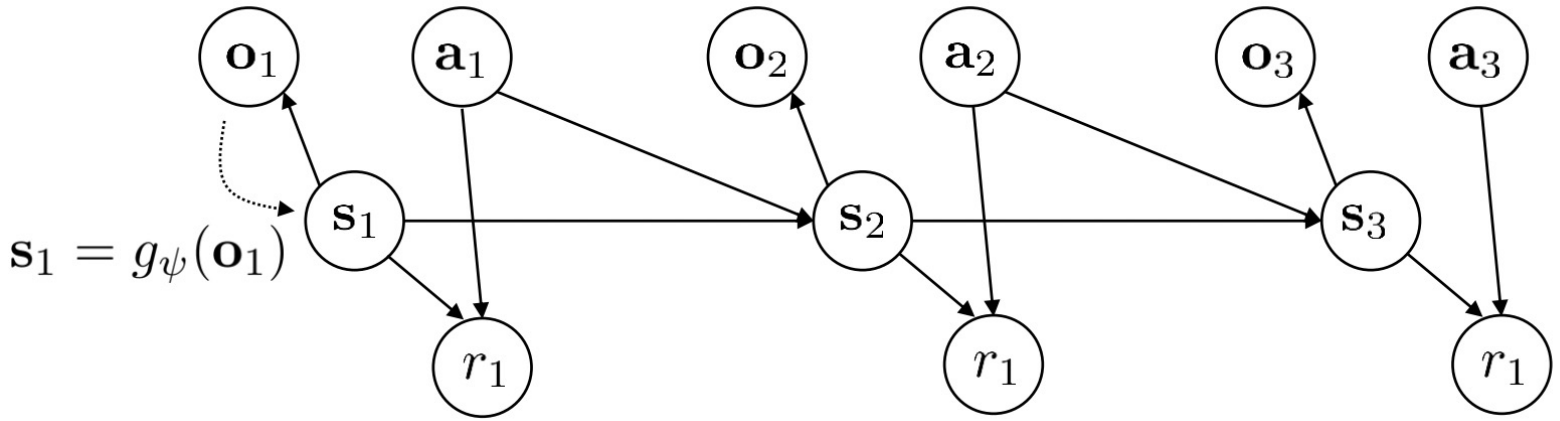
$$q_\psi(\mathbf{s}_t|\mathbf{o}_t) = \delta(\mathbf{s}_t = g_\psi(\mathbf{o}_t)) \Rightarrow \mathbf{s}_t = g_\psi(\mathbf{o}_t)$$

- Now everything becomes differentiable and can train with backprop

$$\max_{\phi,\psi} \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \log p_\phi(g_\psi(\mathbf{o}_{t+1,i})|g_\psi(\mathbf{o}_{t,i}), \mathbf{a}_{t,i}) + \log p_\phi(\mathbf{o}_{t,i}|g_\psi(\mathbf{o}_{t,i}))$$

Learning Latent Space Model with Rewards

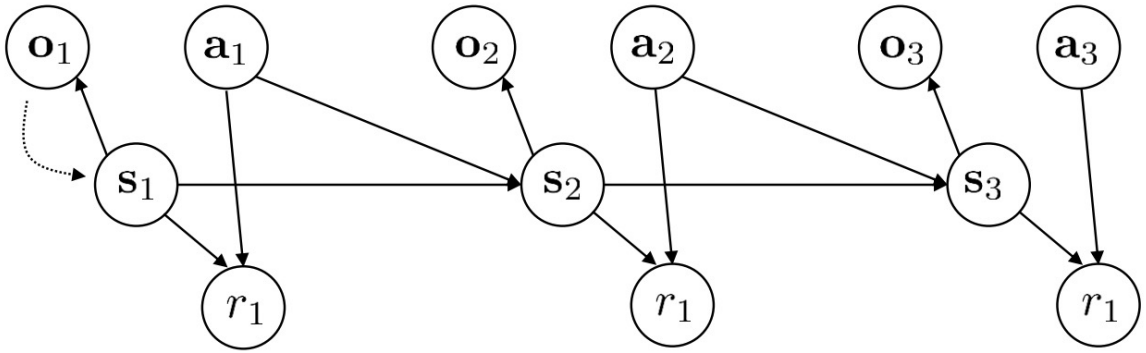
- Since we also do not know the reward function on the latent, we should learn it together



$$\max_{\phi,\psi} \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \log p_\phi(g_\psi(\mathbf{o}_{t+1,i}) | g_\psi(\mathbf{o}_{t,i}), \mathbf{a}_{t,i}) + \log p_\phi(\mathbf{o}_{t,i} | g_\psi(\mathbf{o}_{t,i})) + \log p_\phi(r_{t,i} | g_\psi(\mathbf{o}_{t,i}))$$

latent space dynamics | image reconstruction | reward model

Model-Based RL with Latent Space Model



model-based reinforcement learning with latent state:

1. run base policy $\pi_0(\mathbf{a}_t|\mathbf{o}_t)$ (e.g., random policy) to collect $\mathcal{D} = \{(\mathbf{o}, \mathbf{a}, \mathbf{o}')_i\}$
2. learn $p_\phi(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$, $p_\phi(r_t|\mathbf{s}_t)$, $p(\mathbf{o}_t|\mathbf{s}_t)$, $g_\psi(\mathbf{o}_t)$
3. plan through the model to choose actions
4. execute the first planned action, observe resulting $\mathbf{o}'$ (MPC)
5. append $(\mathbf{o}, \mathbf{a}, \mathbf{o}')$ to dataset $\mathcal{D}$

every N steps

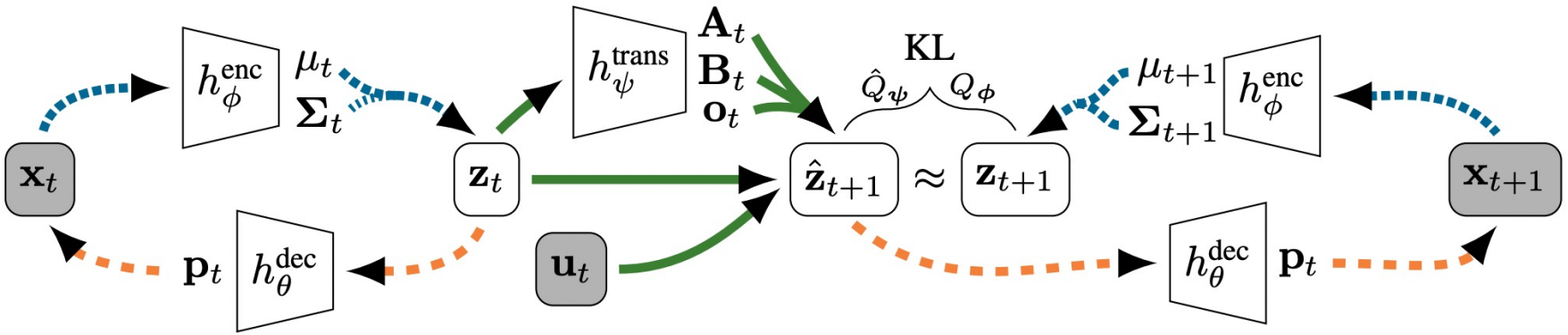
Model-Based RL with Latent Space Model

- Example: Embed to control

Embed to Control: A Locally Linear Latent Dynamics Model for Control from Raw Images

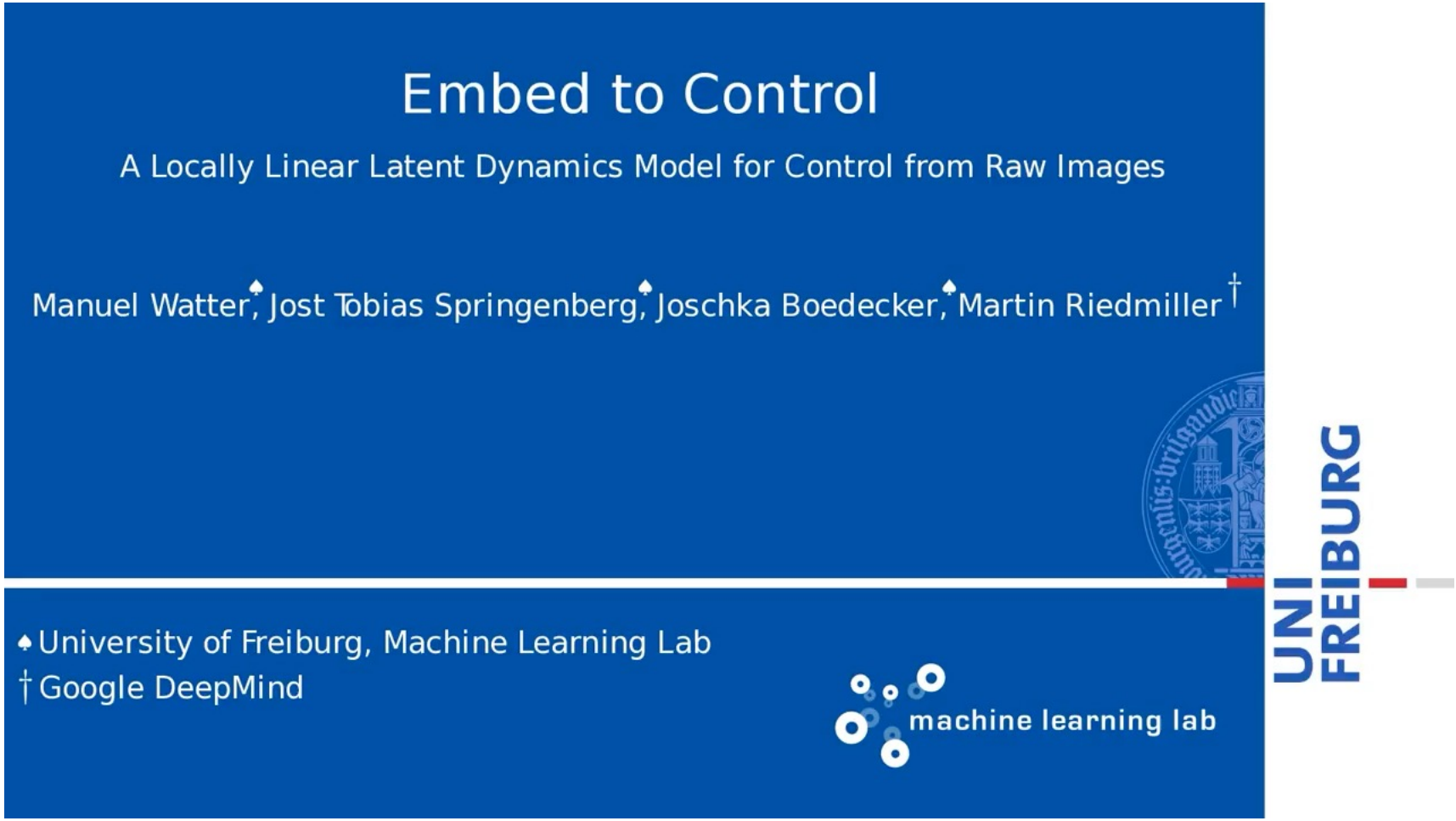
Manuel Watter* **Jost Tobias Springenberg***
Joschka Boedecker
University of Freiburg, Germany
{watterm, springj, jboedeck}@cs.uni-freiburg.de

Martin Riedmiller
Google DeepMind
London, UK
riedmiller@google.com



Model-Based RL with Latent Space Model

- Embed to control



Model-Based RL with Latent Space Model

- Dreamer

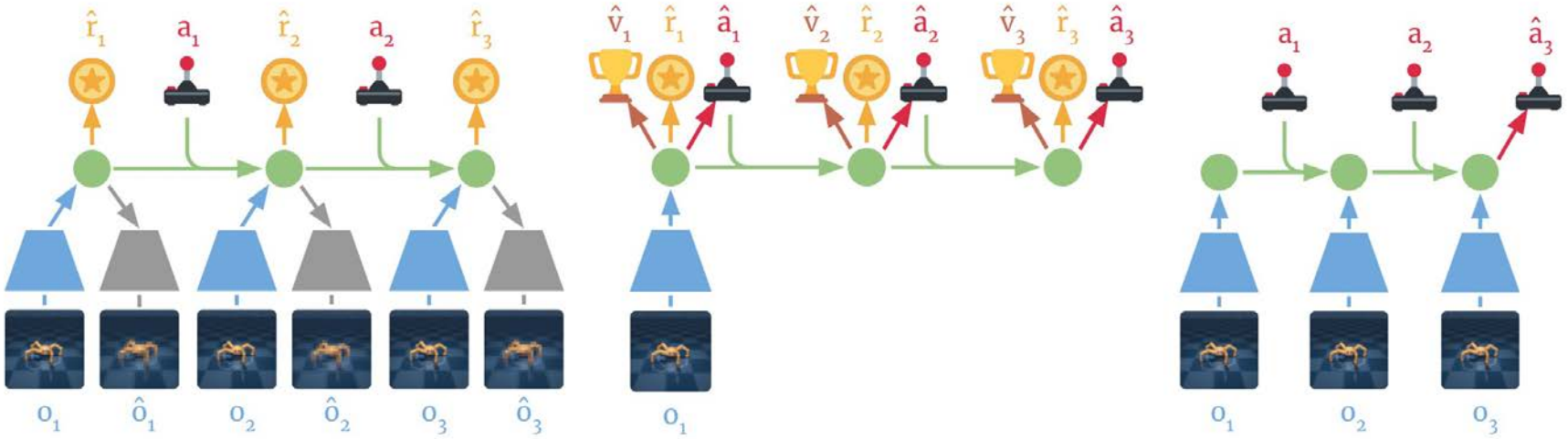
DREAM TO CONTROL: LEARNING BEHAVIORS BY LATENT IMAGINATION

Danijar Hafner *
University of Toronto
Google Brain

Timothy Lillicrap
DeepMind

Jimmy Ba
University of Toronto

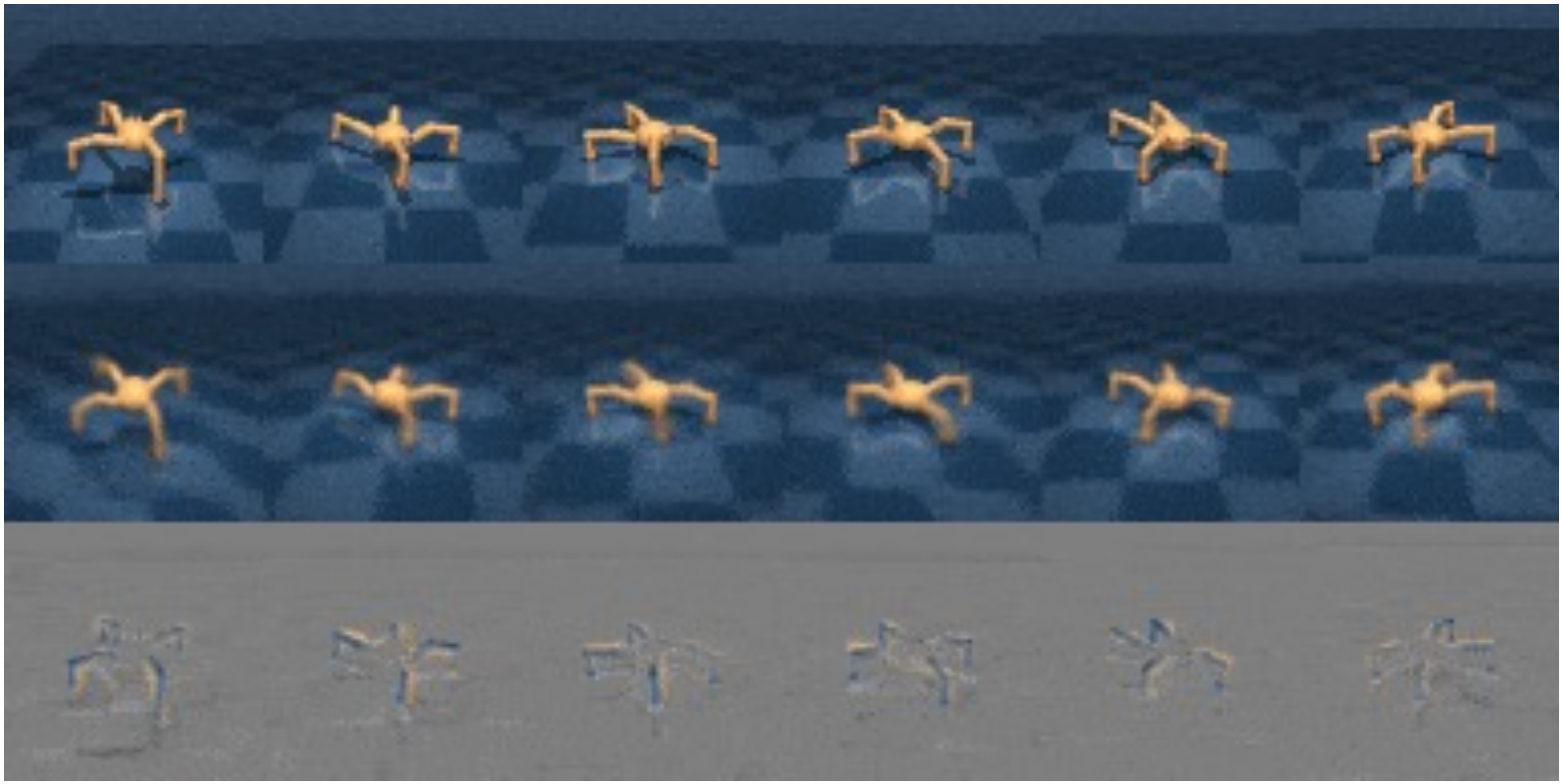
Mohammad Norouzi
Google Brain



(a) Learn dynamics from experience (b) Learn behavior in imagination (c) Act in the environment

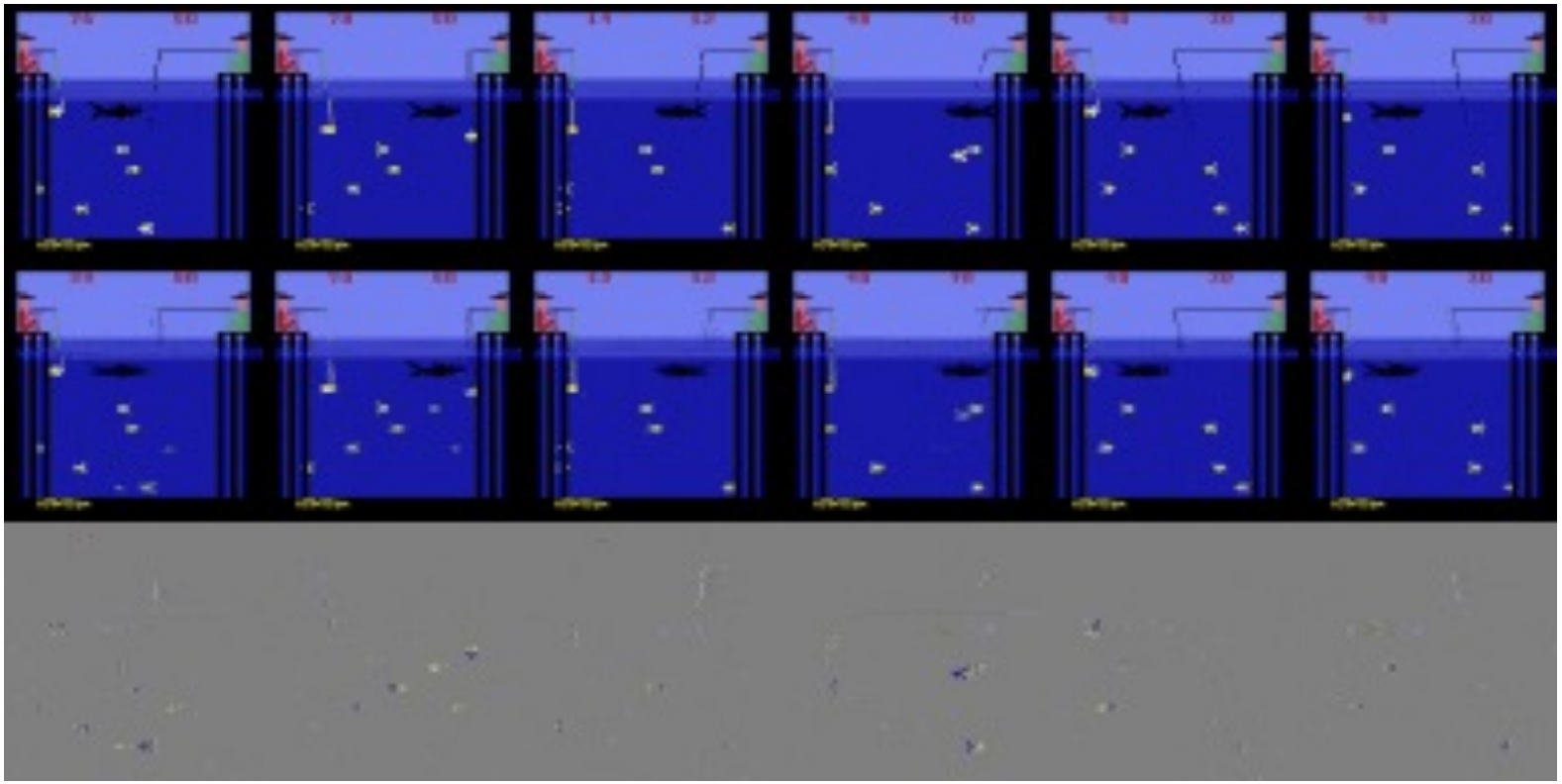
Model-Based RL with Latent Space Model

- Dreamer prediction from learned model



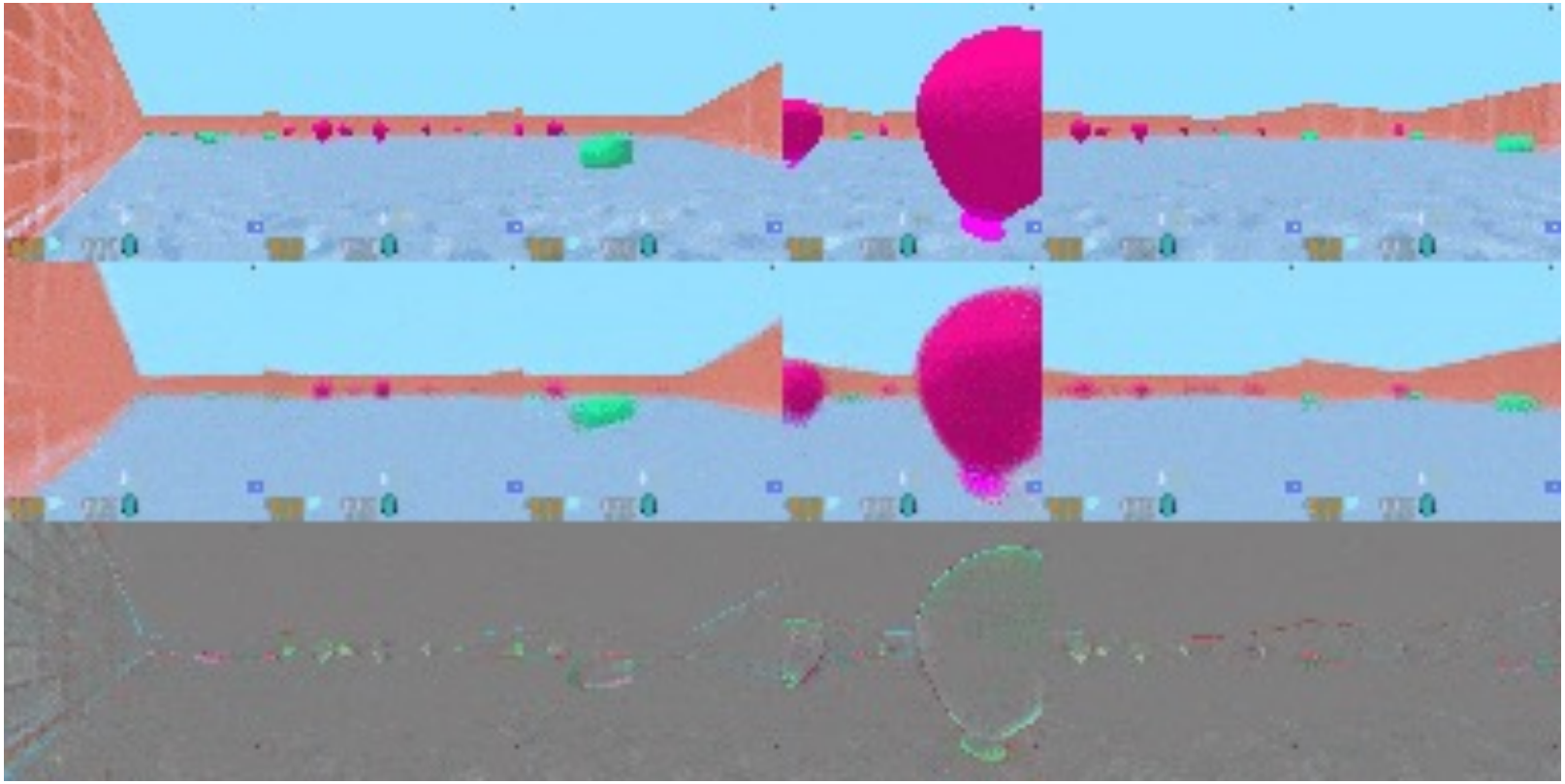
Model-Based RL with Latent Space Model

- Dreamer prediction from learned model



Model-Based RL with Latent Space Model

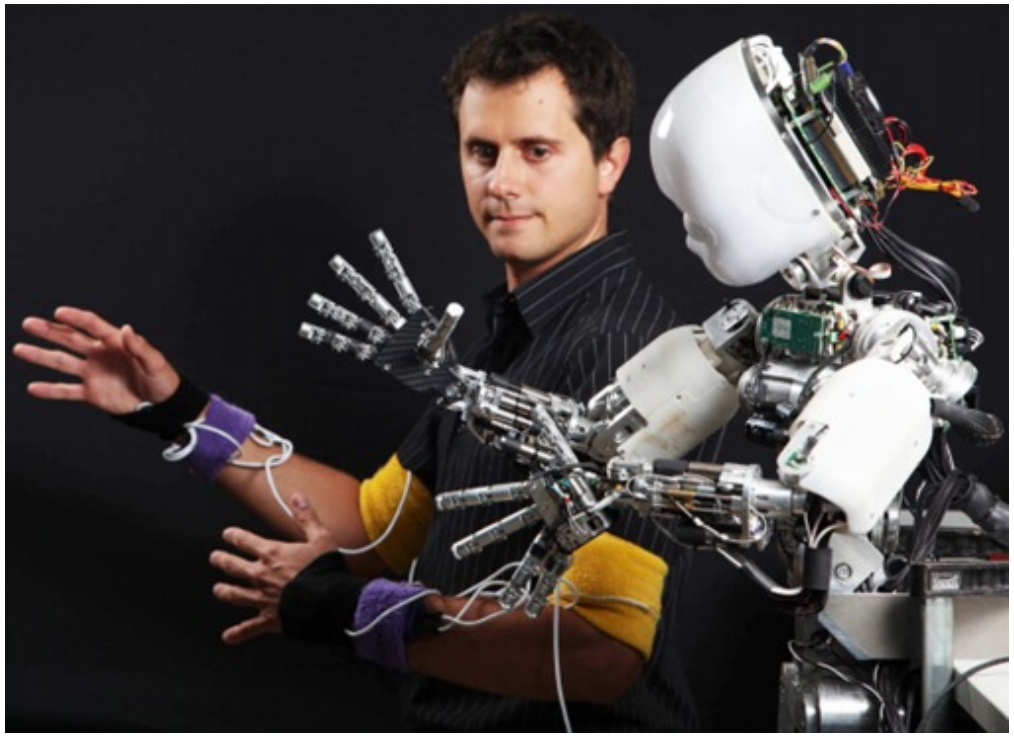
- Dreamer prediction from learned model



Contents

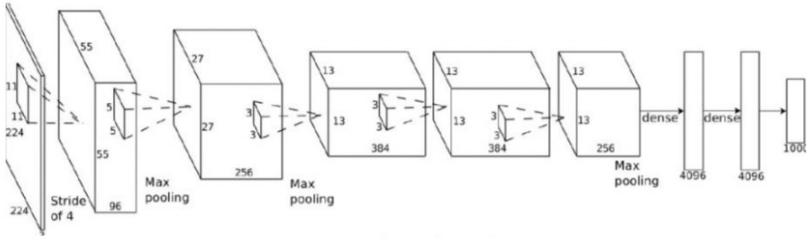
Imitation learning

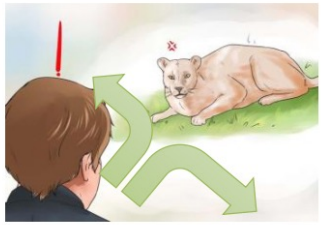
- Human does not purely learn from “trial and error”
- We learn from teachers
- For robots, we can also make them learn from demonstrations



Imitation learning formulation

- A complete formulation for imitation learning

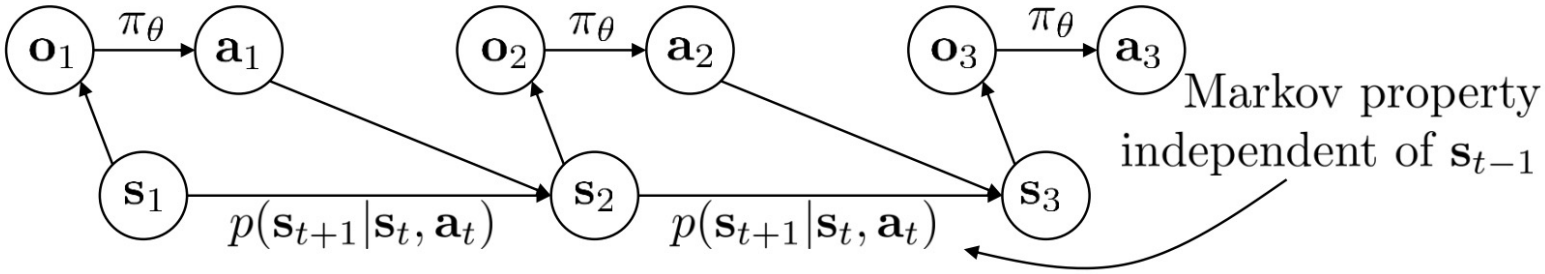




$\mathbf{o}_t$ $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$ $\mathbf{a}_t$

$\mathbf{s}_t$ – state
$\mathbf{o}_t$ – observation
$\mathbf{a}_t$ – action

$\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$ – policy
$\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ – policy (fully observed)



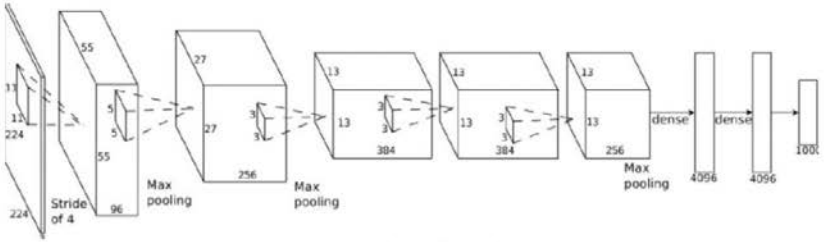
Imitation learning algorithms

- What practical imitation learning algorithms we have?
  - Behavior cloning
  - Learn from demonstration with RL
  - Inverse RL

Behavior cloning

- The most basic setting for learning with demonstrations is behavior cloning

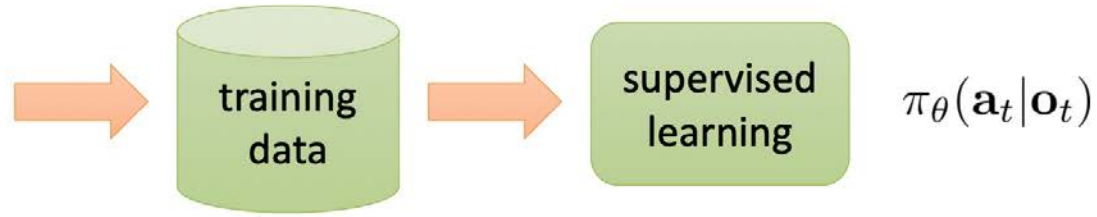
$\mathbf{o}_t$



$\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$

$\mathbf{a}_t$

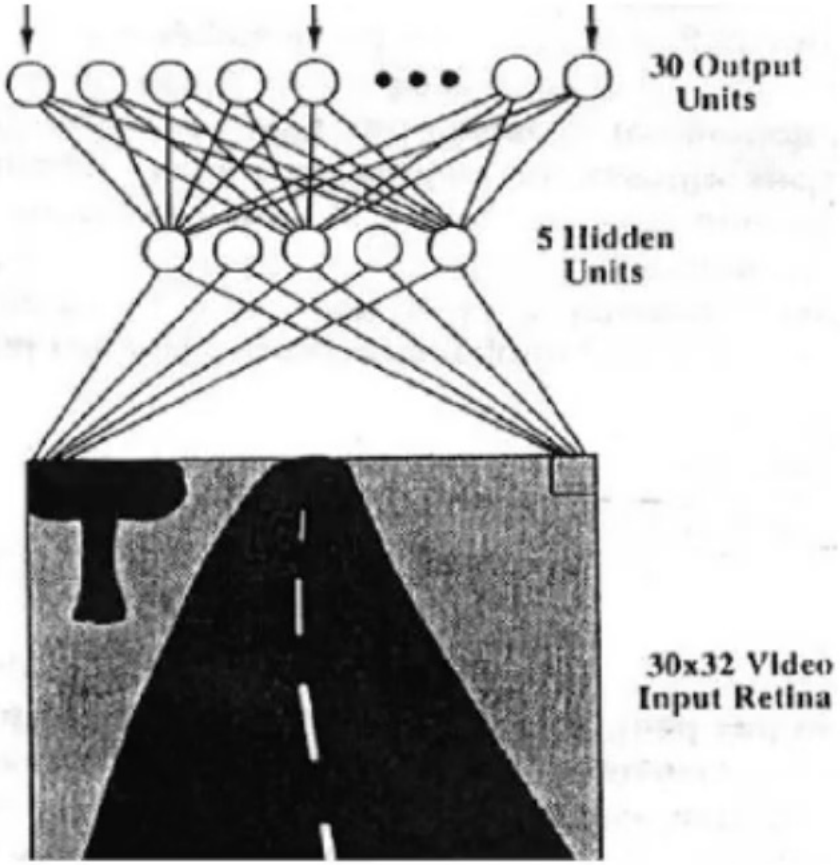
$\mathbf{o}_t$
$\mathbf{a}_t$



$\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$

History: Deep Behavior Cloning for Driving

- ALVINN: Autonomous Land Vehicle In a Neural Network

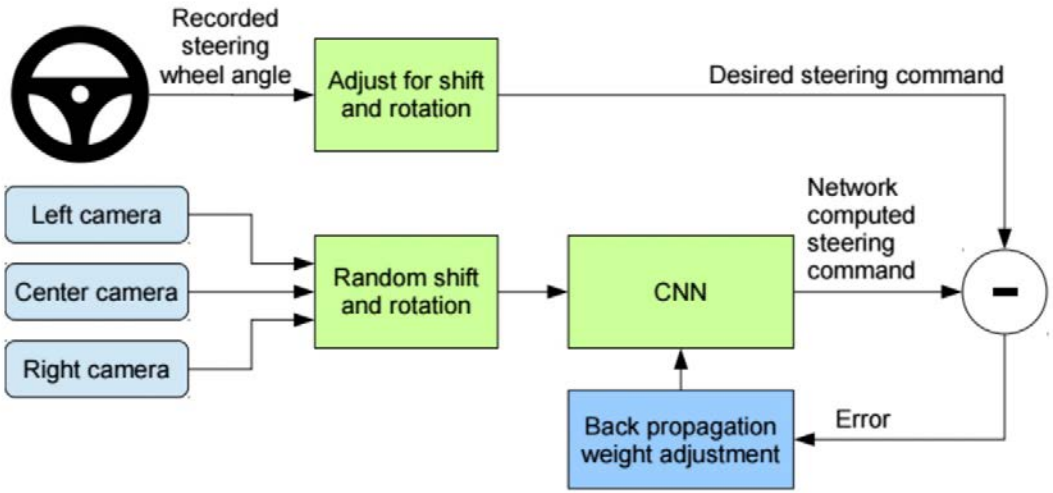


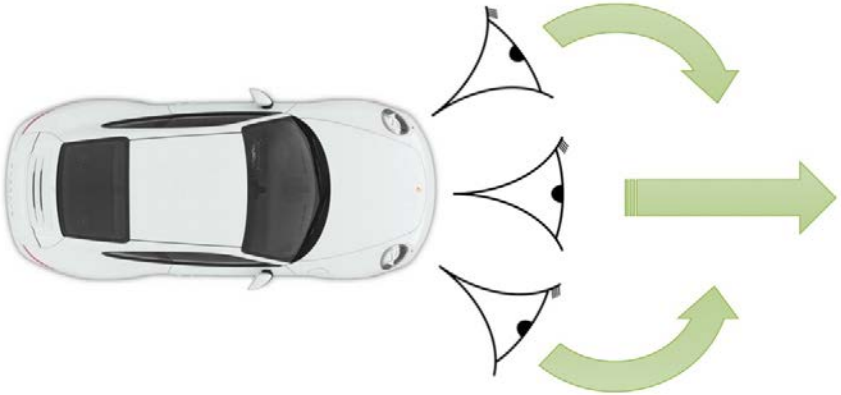
History: Deep Behavior Cloning for Driving

- ALVINN: Autonomous Land Vehicle In a Neural Network

End-to-end driving via imitation learning

- Behavior cloning with deep learning

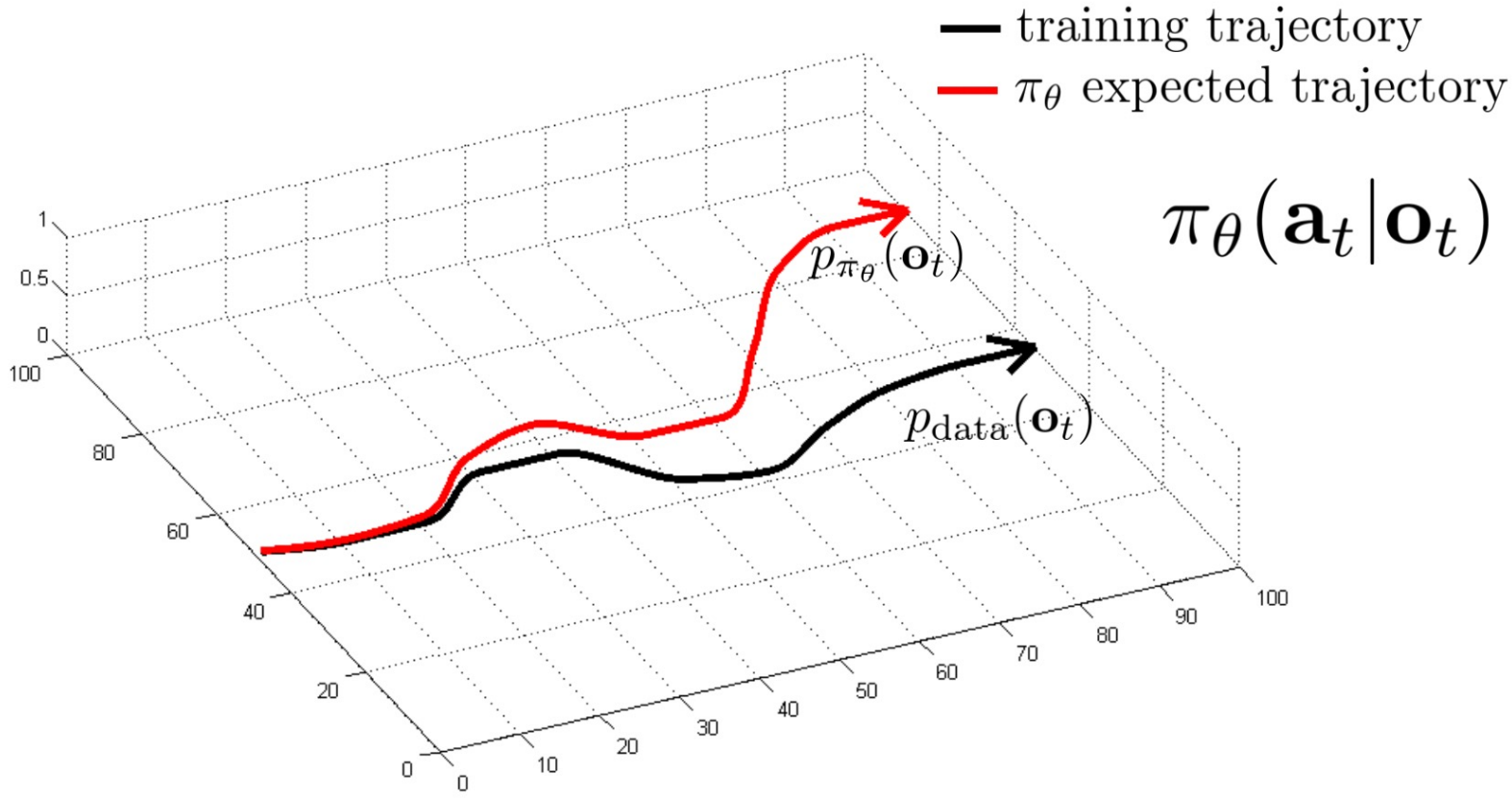




End-to-end driving via imitation learning

Problems for behavior cloning

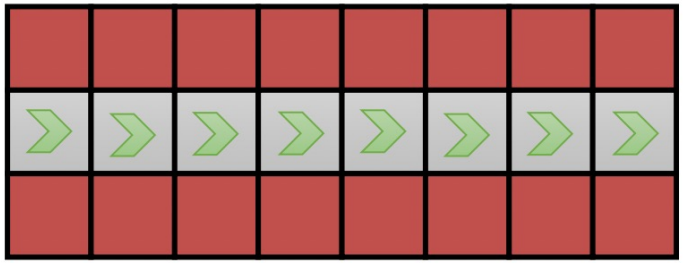
- Any problems?



- There is distributional shift

Problems for behavior cloning

- The basic behavior cloning is like wire-walking: if make one mistake and you will then fail



- How to address the problem?
  - Data aggregation
  - Better model to fit the expert behavior

Data aggregation

- Dagger

DAgger: Dataset Aggregation

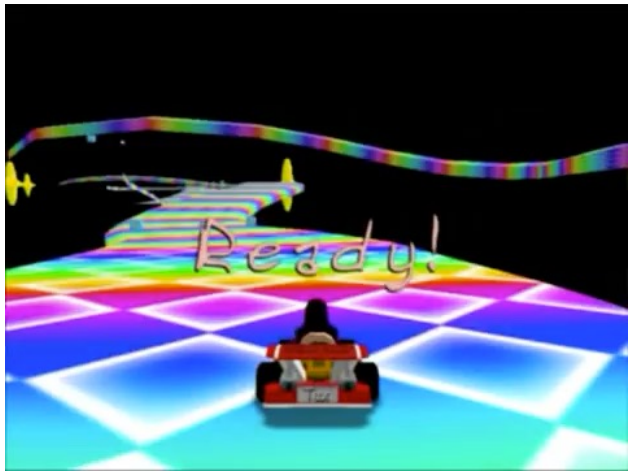
goal: collect training data from $p_{\pi_\theta}(\mathbf{o}_t)$ instead of $p_{\text{data}}(\mathbf{o}_t)$

how? just run $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$

but need labels $\mathbf{a}_t$!

1. train $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$ from human data $\mathcal{D} = \{\mathbf{o}_1, \mathbf{a}_1, \ldots, \mathbf{o}_N, \mathbf{a}_N\}$
2. run $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$ to get dataset $\mathcal{D}_\pi = \{\mathbf{o}_1, \ldots, \mathbf{o}_M\}$
3. Ask human to label $\mathcal{D}_\pi$ with actions $\mathbf{a}_t$
4. Aggregate: $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_\pi$

Data aggregation



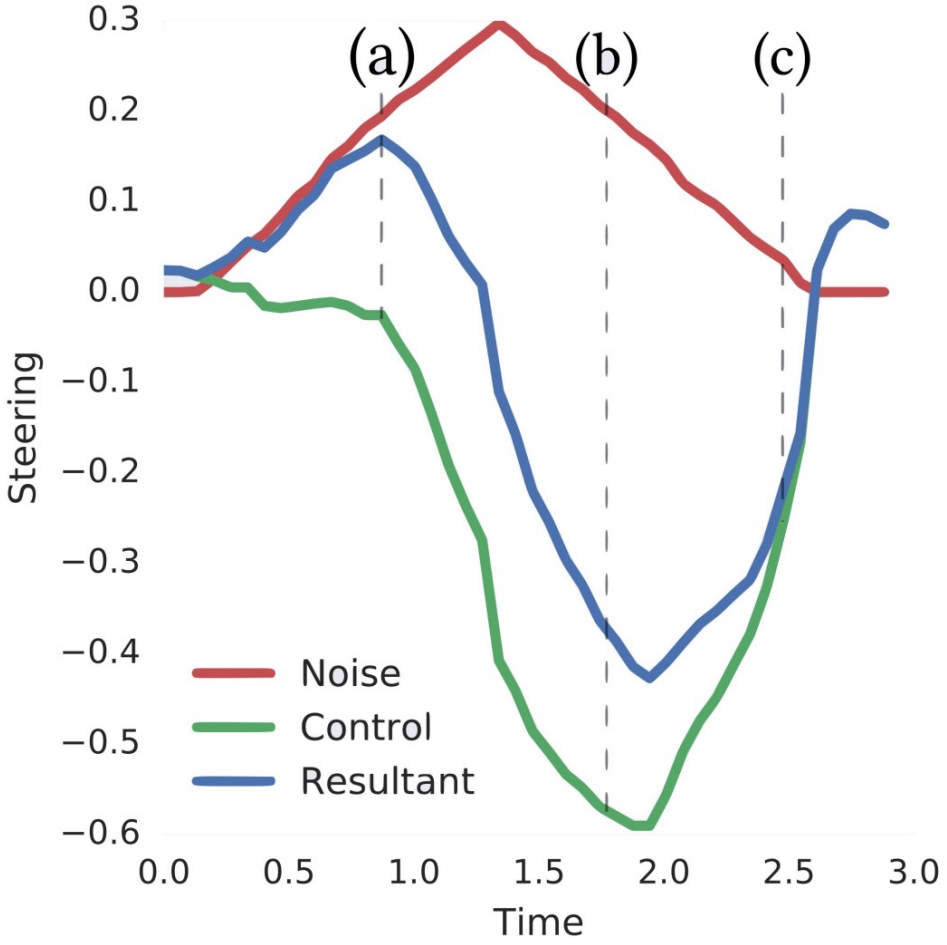
Expert

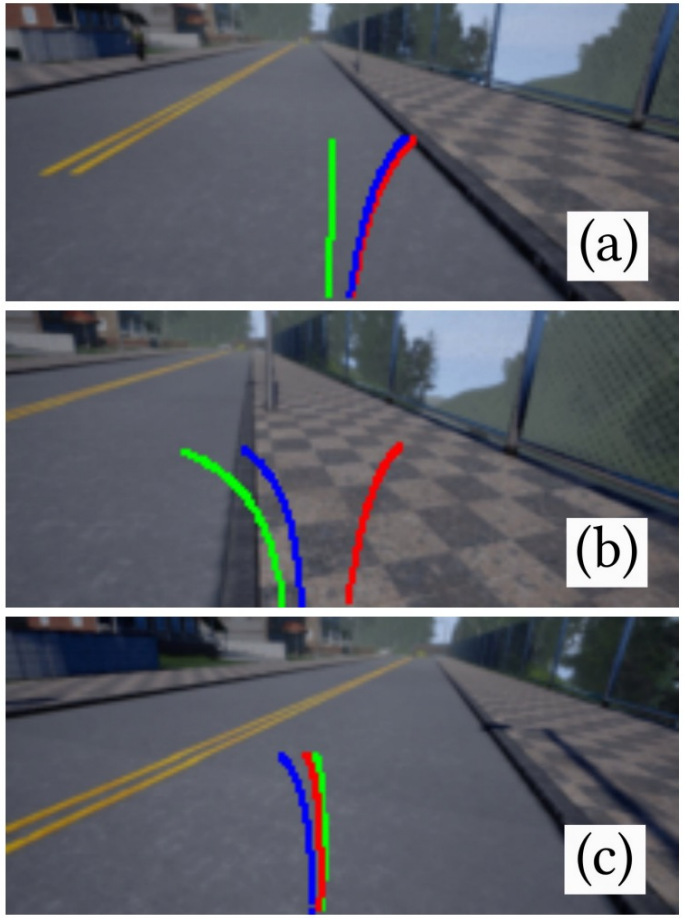
w/o DAgger

w/ DAgger

End-to-end driving via imitation learning

- Data augmentation by adding noises

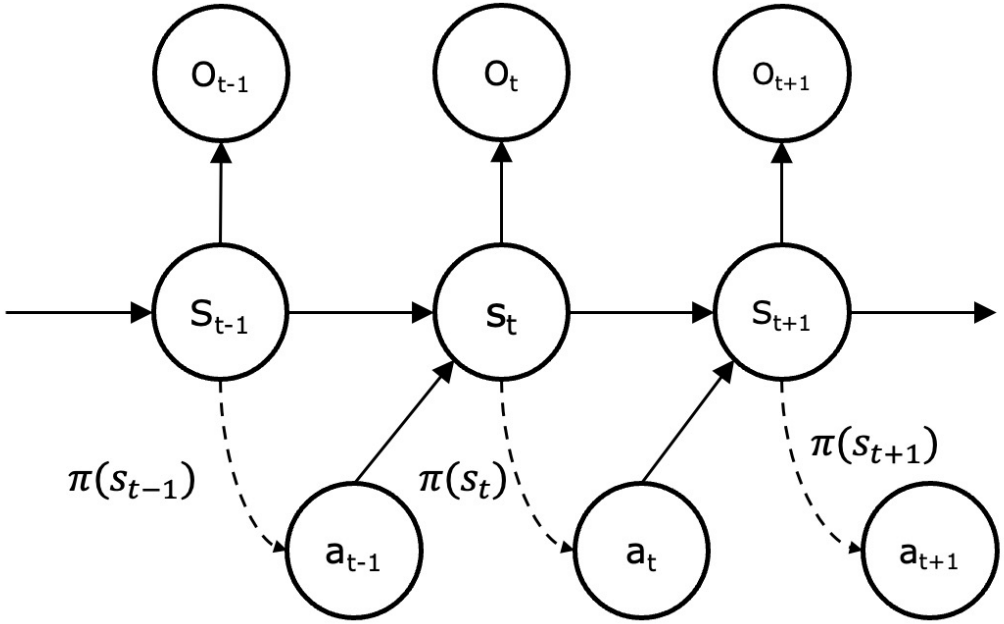




End-to-end driving via imitation learning

Adding historical observations

- Policy conditioned on current observation does not obey the Markov property



$$\pi_\theta(\mathbf{a}_t|\mathbf{o}_t) \quad \Rightarrow \quad \pi_\theta(\mathbf{a}_t|\mathbf{o}_1, \ldots, \mathbf{o}_t)$$

behavior depends only on current observation

behavior depends on all past observations

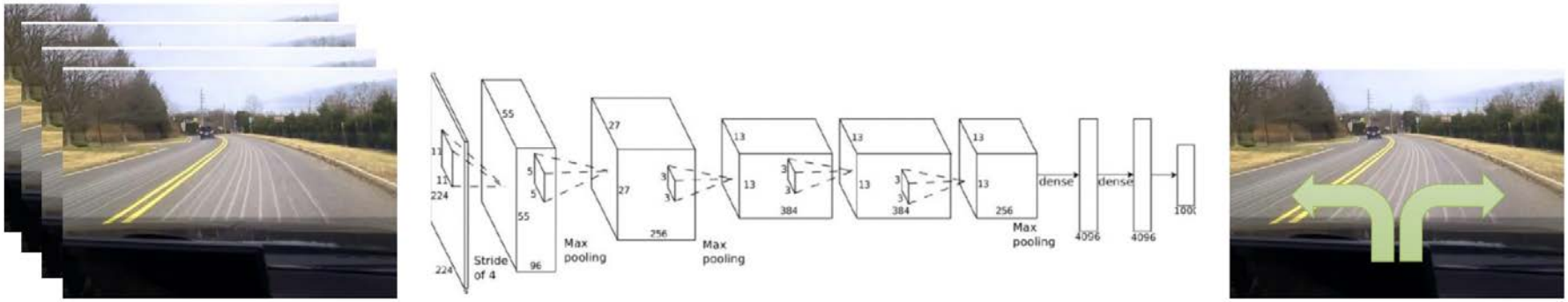
Adding historical observations

- Policy conditioned on historical observations obey the Markov property

$$\pi_\theta(\mathbf{a}_t|\mathbf{o}_t) \quad \Rightarrow \quad \pi_\theta(\mathbf{a}_t|\mathbf{o}_1, \ldots, \mathbf{o}_t)$$

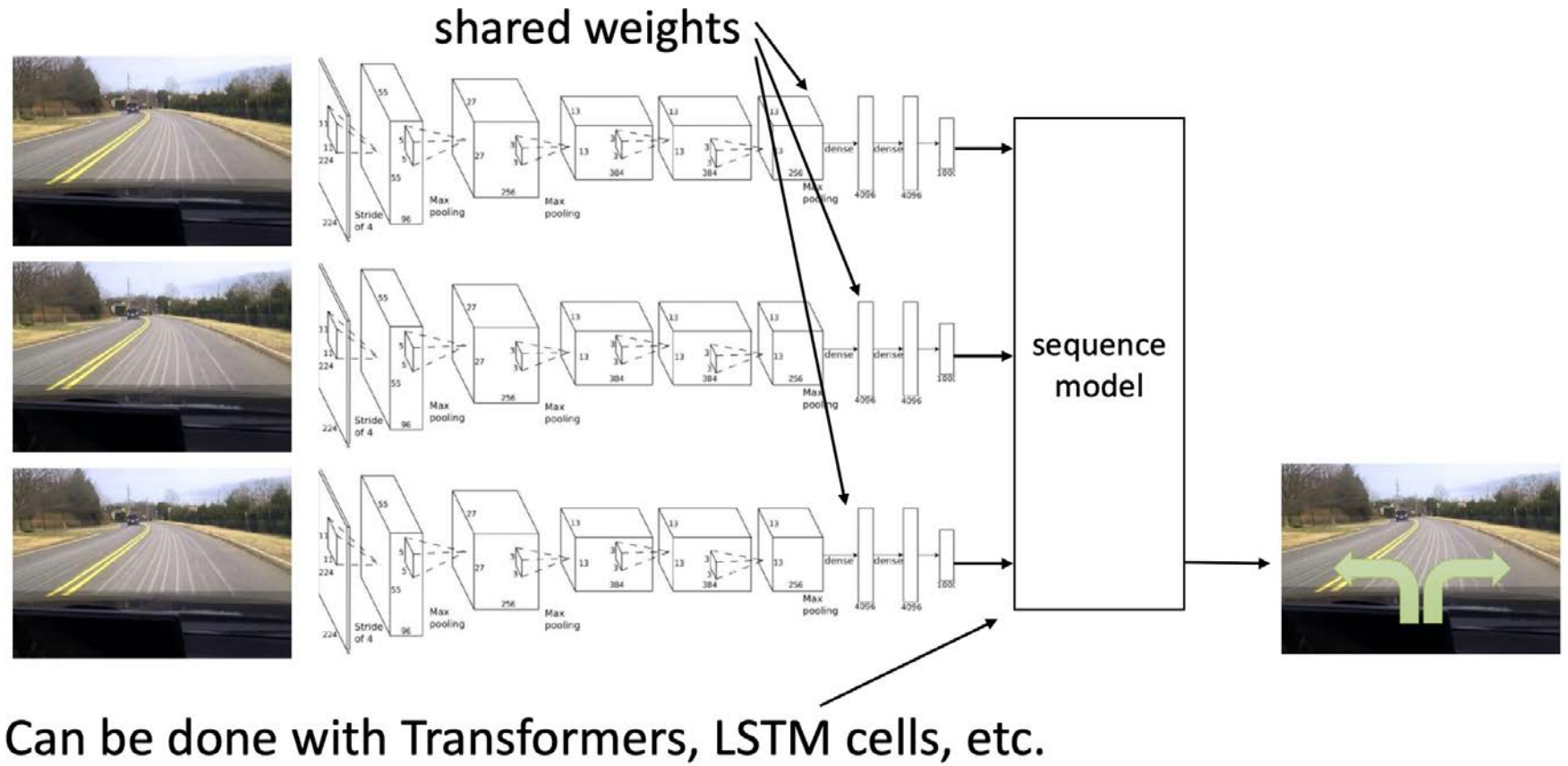
behavior depends only on current observation

behavior depends on all past observations



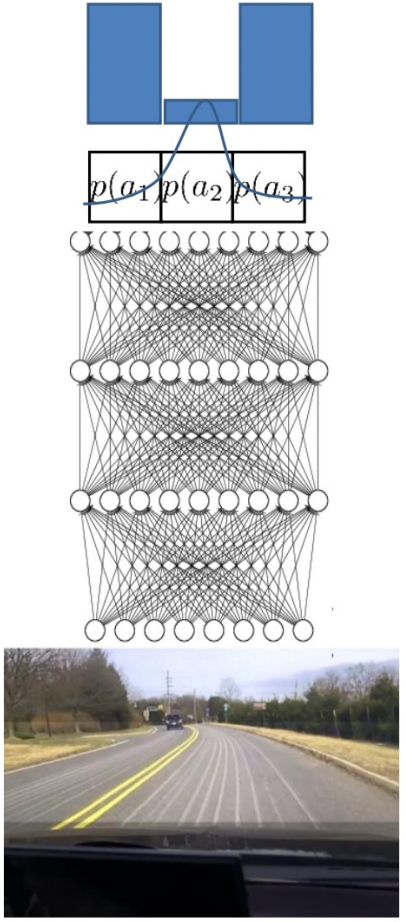
Adding historical observations

- History conditioned policy with sequence model



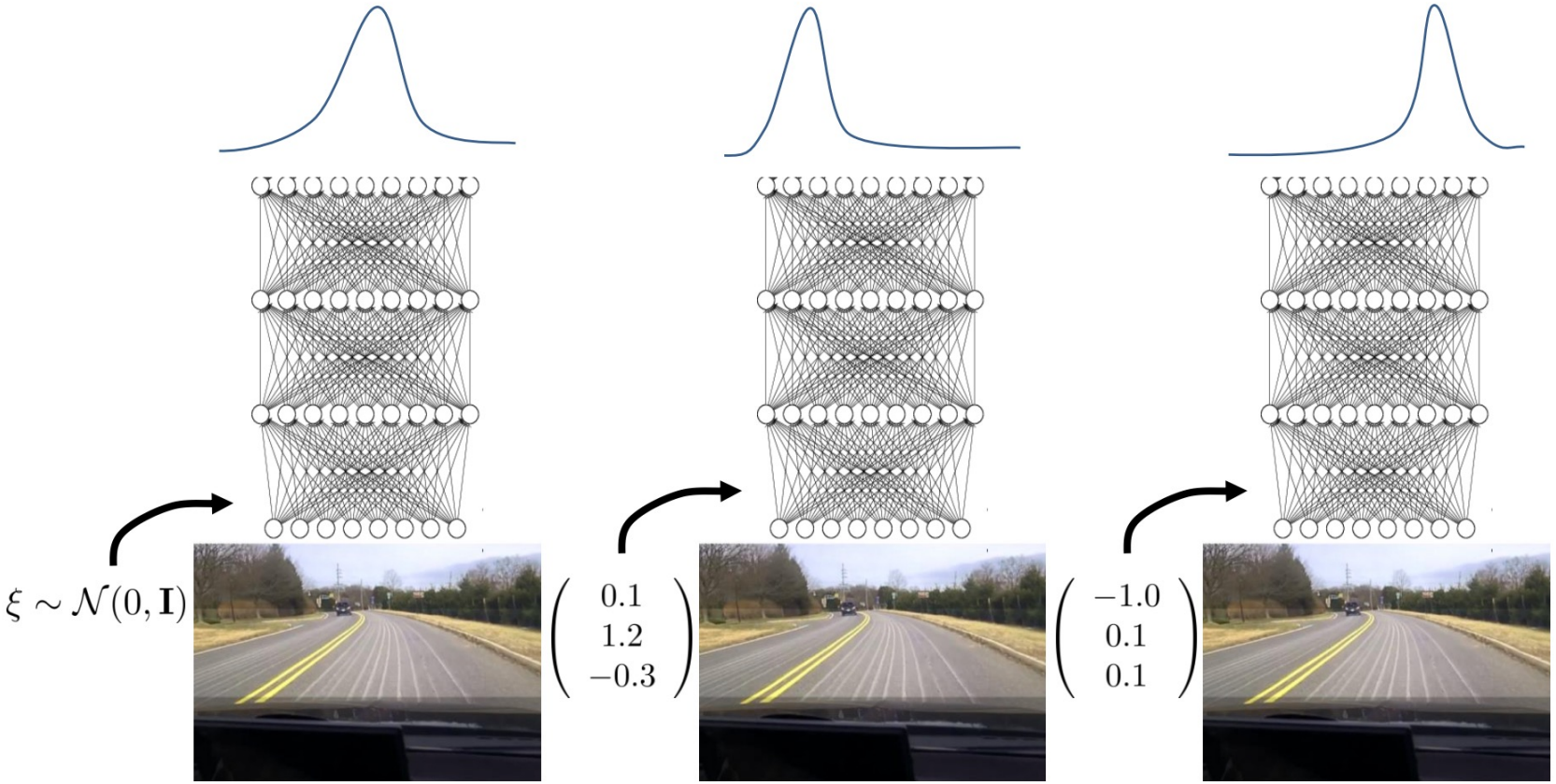
Learn multi-modal behavior

- The expert behaviors are mostly multi-modal with uncertainties



Latent variable models

- Learn latent variable models which generates uncertainty



DDPG from Demonstration (DDPGfD)

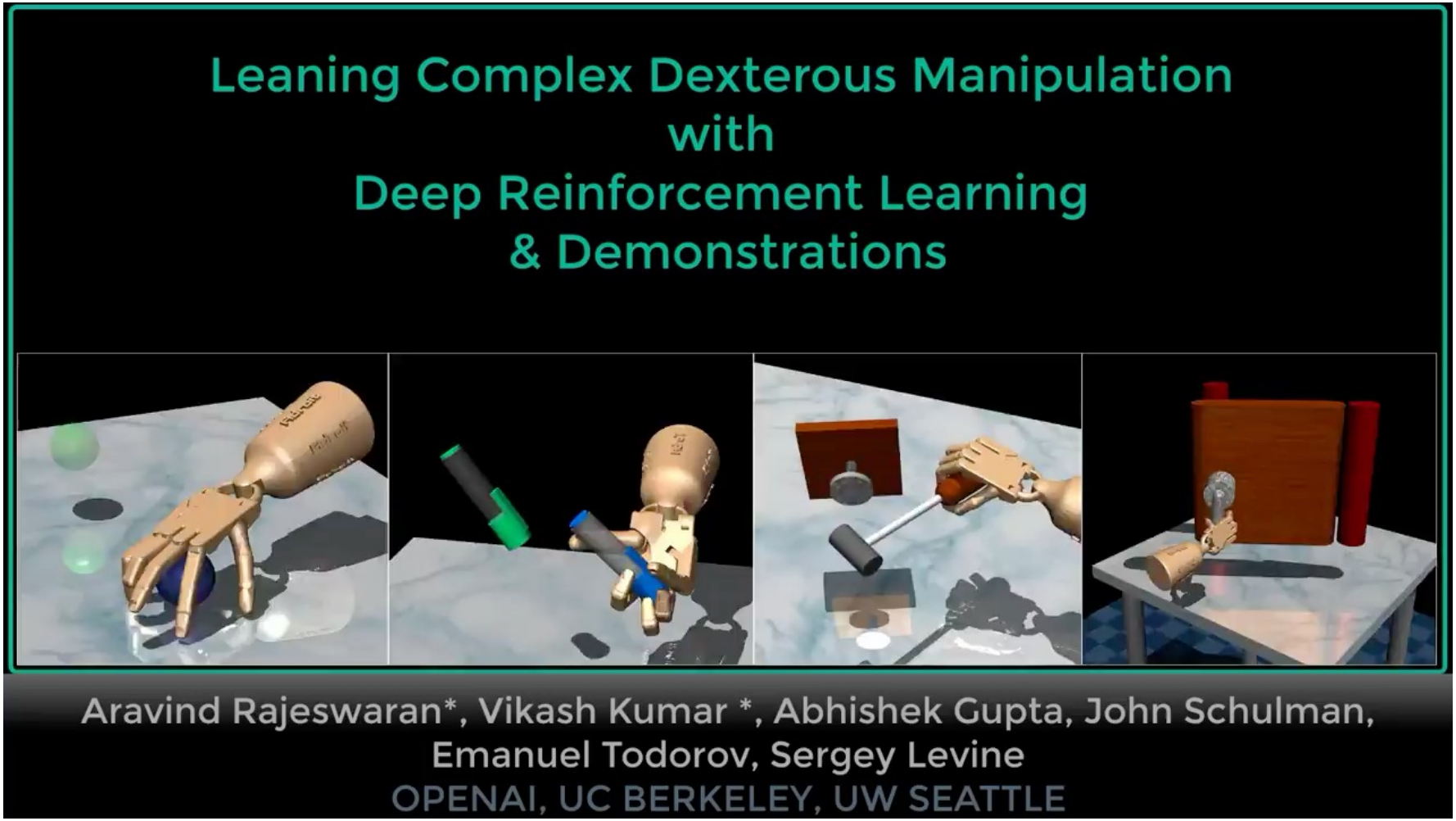
Algorithm 1 Robust DDPGfD

Require: policy parameter θ, target policy parameter θ', Q network parameter ϕ, target Q network parameter ϕ'

1: **for** $t \in [0, \textit{episode\_length}]$ **do**
2: Collect demonstration, save to replay buffer $\mathcal{B}$
3: Select an action $a_t = \pi_\theta(s_t)$ ~~$+\sigma$, where $\sigma \sim \mathcal{N}(0, \Sigma)$~~
4: Take action a_t, observe(s_t, a_t, s_{t+1}, r_t), add it to $\mathcal{B}$
5: On-policy correction a'_t if necessary, overriding a_t
6: Compute TD target with minibatch data from $\mathcal{B}$ and target networks $Q_{\phi'}$ and $\pi_{\theta'}$
7: Update θ and pi with the TD target
8: Update target networks θ' and ϕ' (Polyak averaging)
9: **end for**

DDPG from Demonstration (DDPGfD)

- Dexterous manipulation



DDPG from Demonstration (DDPGfD)

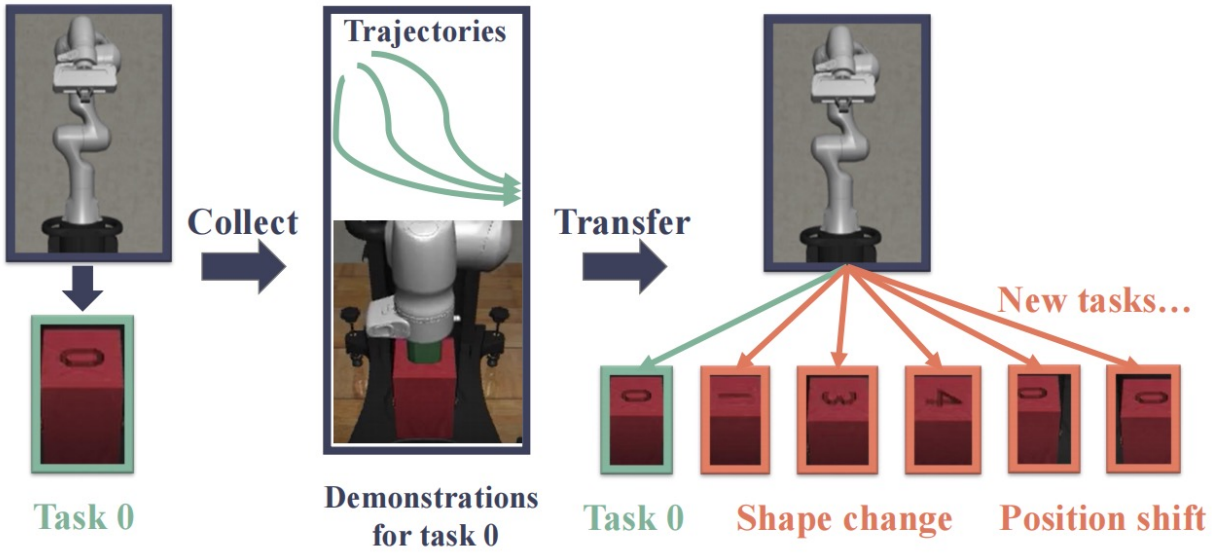
- Robot insertion

LfD from mismatched demonstrations

- Existing methods require collecting demos for each task
- Sparse reward RL with transferable demonstrations for task generalization

Reinforcement learning with Demonstrations from Mismatched Task under Sparse Reward

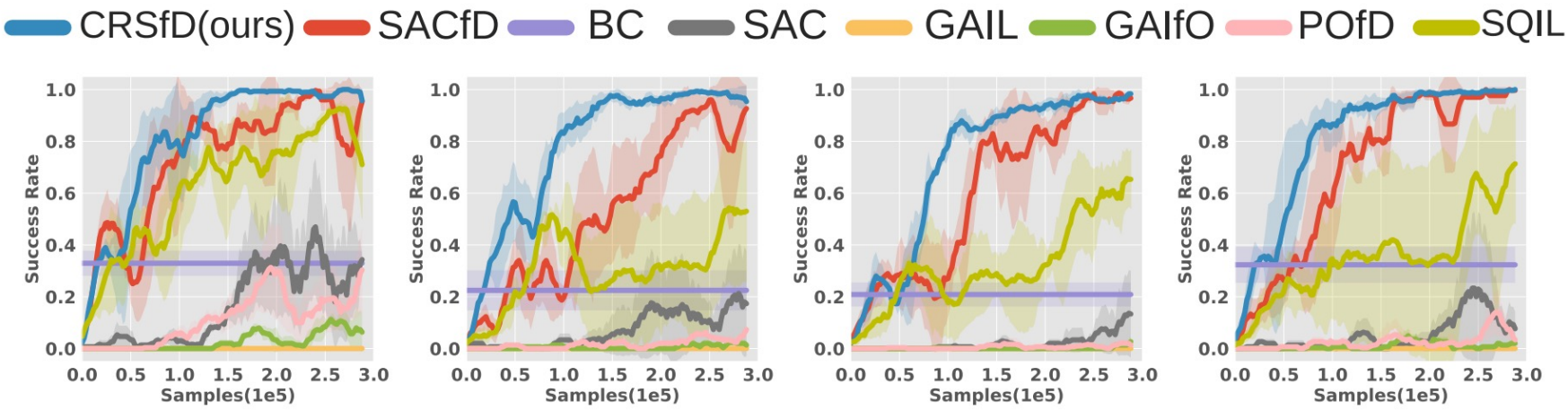
Yanjiang Guo[1], Jingyue Gao[1], Zheng Wu[2], Chengming Shi[1], Jianyu Chen[1,3,*]



LfD from mismatched demonstrations

- Conservatively shapes sparse reward in new tasks to guide exploration

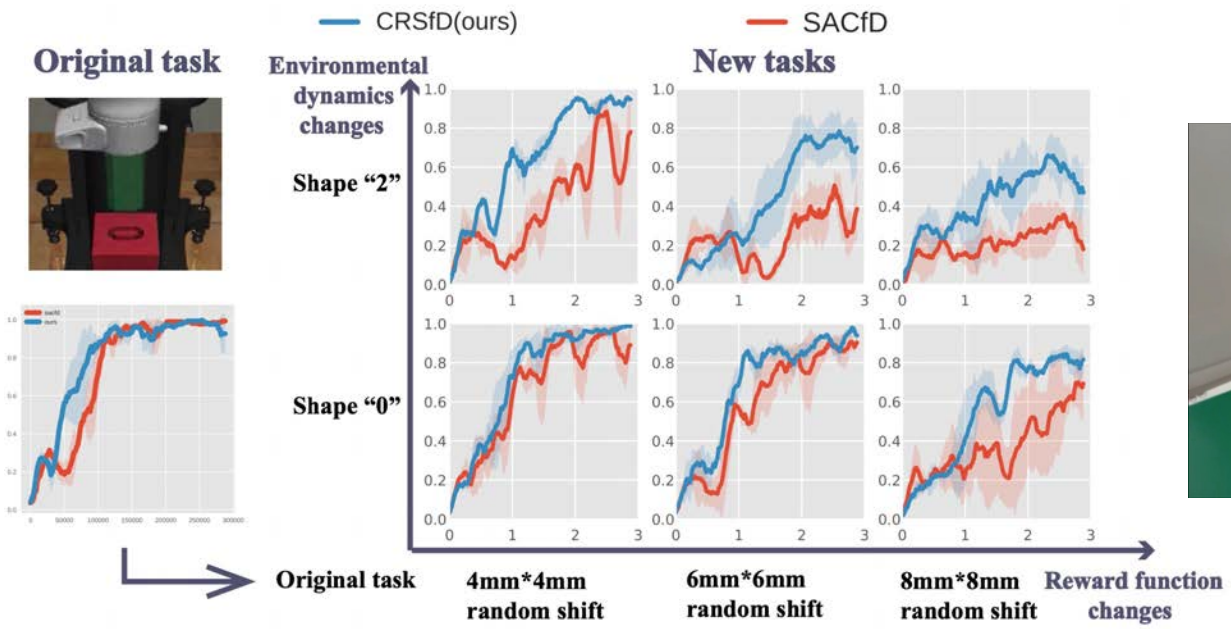
$$R'_i(s,a,s') = R_i(s,a,s') + \gamma \widetilde{V}^D_{M_0}(s') - \widetilde{V}^D_{M_0}(s)$$



(a) Hole "1" (b) Hole "2" (c) Hole "3" (d) Hole "4"

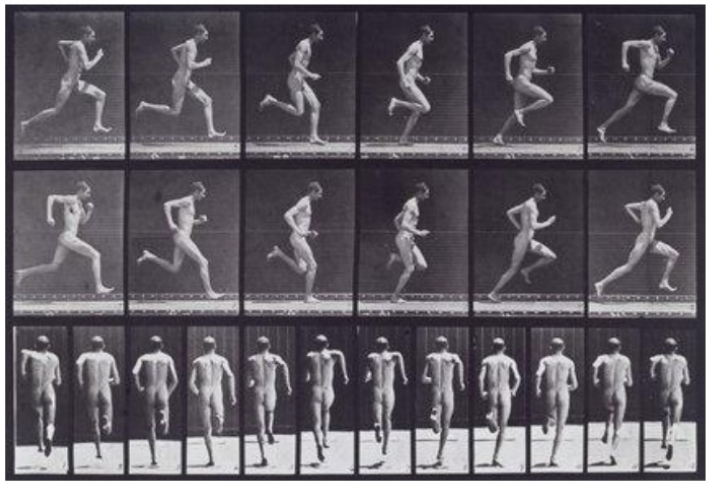
LfD from mismatched demonstrations

- Better performance when transferring to different tasks



Human behavior

- Optimal control as a model of human behavior



Muybridge (c. 1870)

Mombaur et al. '09

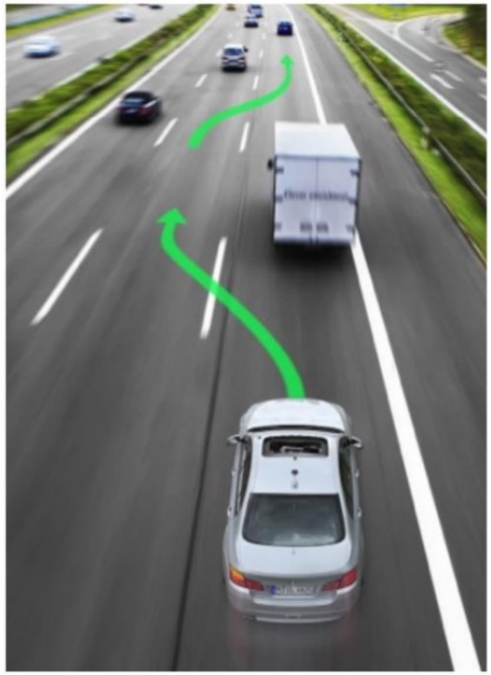
$$\mathbf{a}_1, \ldots, \mathbf{a}_T = \arg \max_{\mathbf{a}_1, \ldots, \mathbf{a}_T} \sum_{t=1}^{T} r(\mathbf{s}_t, \mathbf{a}_t)$$

Human learning

- Human learning is different with the above imitation learning
- Imitation learning
  - Copy the actions performed by the expert
  - No reasoning about outcomes of actions
- Human learning
  - Copy the intent of the expert
  - Might take very different actions!

Inverse Reinforcement Learning

- Assume the data is generated by some optimal agent/controller, and infer its reward



$$r(\mathbf{s}, \mathbf{a})$$

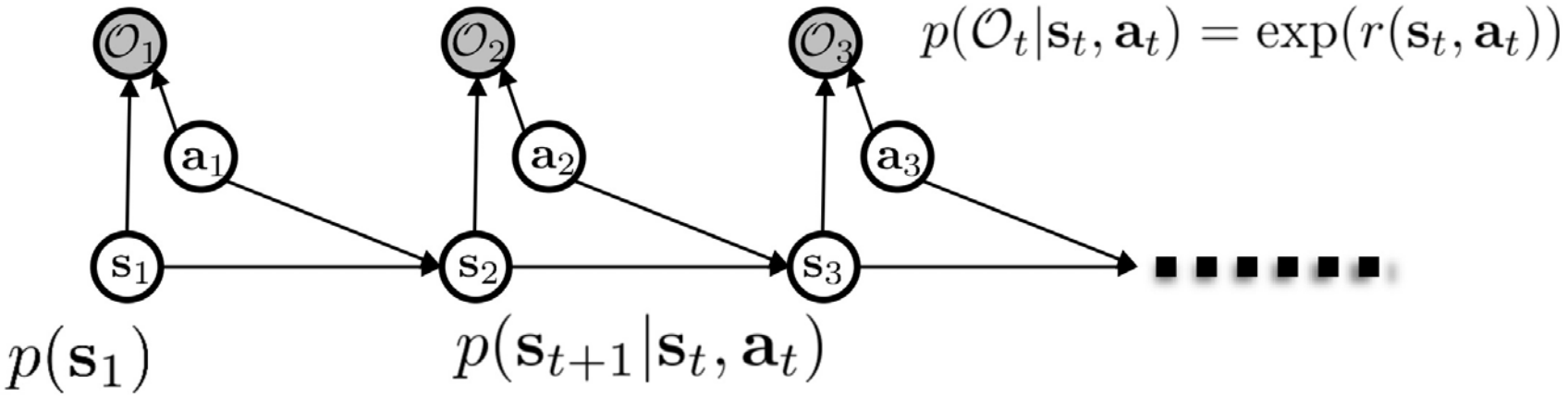
Inverse Reinforcement Learning

- Assume the data is generated by some optimal agent/controller, and infer its reward

"forward" reinforcement learning	inverse reinforcement learning
given:	given:
states $\mathbf{s} \in \mathcal{S}$, actions $\mathbf{a} \in \mathcal{A}$	states $\mathbf{s} \in \mathcal{S}$, actions $\mathbf{a} \in \mathcal{A}$
(sometimes) transitions $p(\mathbf{s}'\|\mathbf{s}, \mathbf{a})$	(sometimes) transitions $p(\mathbf{s}'\|\mathbf{s}, \mathbf{a})$
reward function $r(\mathbf{s}, \mathbf{a})$	samples $\{\tau_i\}$ sampled from $\pi^\star(\tau)$
learn $\pi^\star(\mathbf{a}\|\mathbf{s})$	learn $r_\psi(\mathbf{s}, \mathbf{a})$ (reward parameters)
	...and then use it to learn $\pi^\star(\mathbf{a}\|\mathbf{s})$

Inverse Reinforcement Learning

- Define optimality variables



- The optimal trajectory (data) comes from:

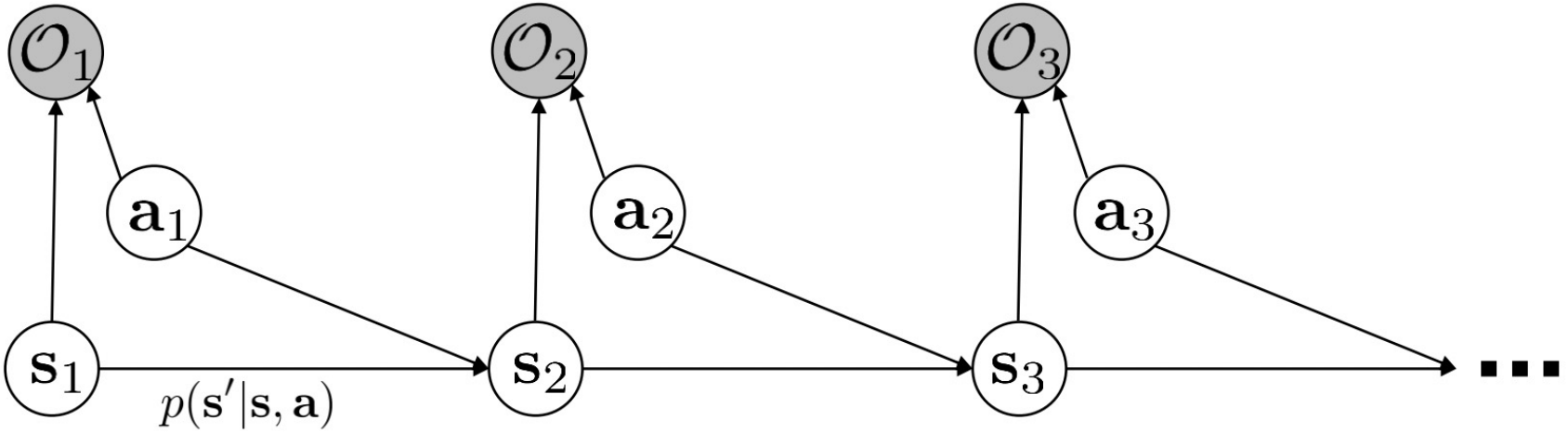
$$p(\mathbf{s}_{1:T}, \mathbf{a}_{1:T}|\mathcal{O}_{1:T})$$

Inverse Reinforcement Learning

$$p(\tau|\mathcal{O}_{1:T}) \qquad p(\mathcal{O}_t|\mathbf{s}_t, \mathbf{a}_t) = \exp(r(\mathbf{s}_t, \mathbf{a}_t))$$

$$p(\tau|\mathcal{O}_{1:T}) = \frac{p(\tau, \mathcal{O}_{1:T})}{p(\mathcal{O}_{1:T})}$$

$$\propto p(\tau) \prod_t \exp(r(\mathbf{s}_t, \mathbf{a}_t)) = p(\tau) \exp\left(\sum_t r(\mathbf{s}_t, \mathbf{a}_t)\right)$$



Inverse Reinforcement Learning

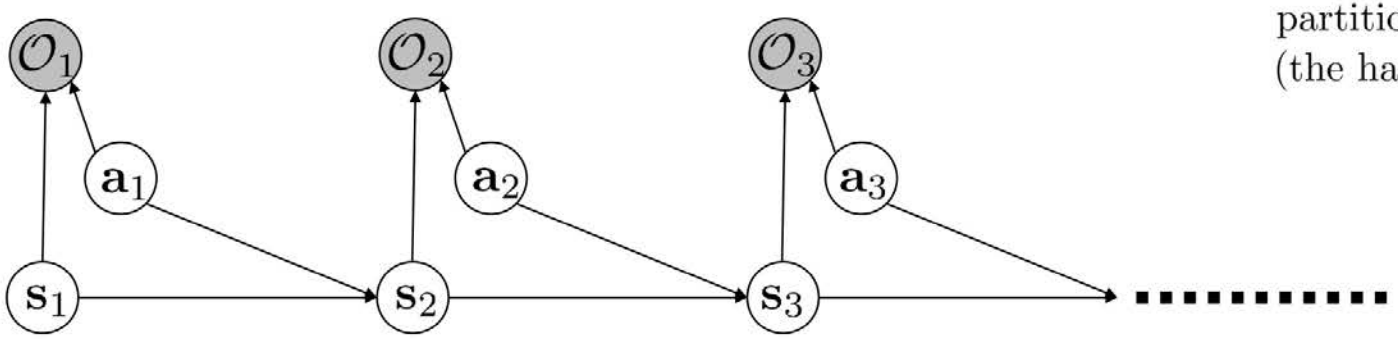
Parameter of reward function

$$p(\tau|\mathcal{O}_{1:T}, \psi) \propto \cancel{p(\tau)} \exp\left(\sum_t r_\psi(\mathbf{s}_t, \mathbf{a}_t)\right)$$

can ignore (independent of ψ)

maximum likelihood learning: $\max_\psi \frac{1}{N} \sum_{i=1}^{N} \log p(\tau_i|\mathcal{O}_{1:T}, \psi) = \max_\psi \frac{1}{N} \sum_{i=1}^{N} r_\psi(\tau_i) - \log Z$

partition function (the hard part)

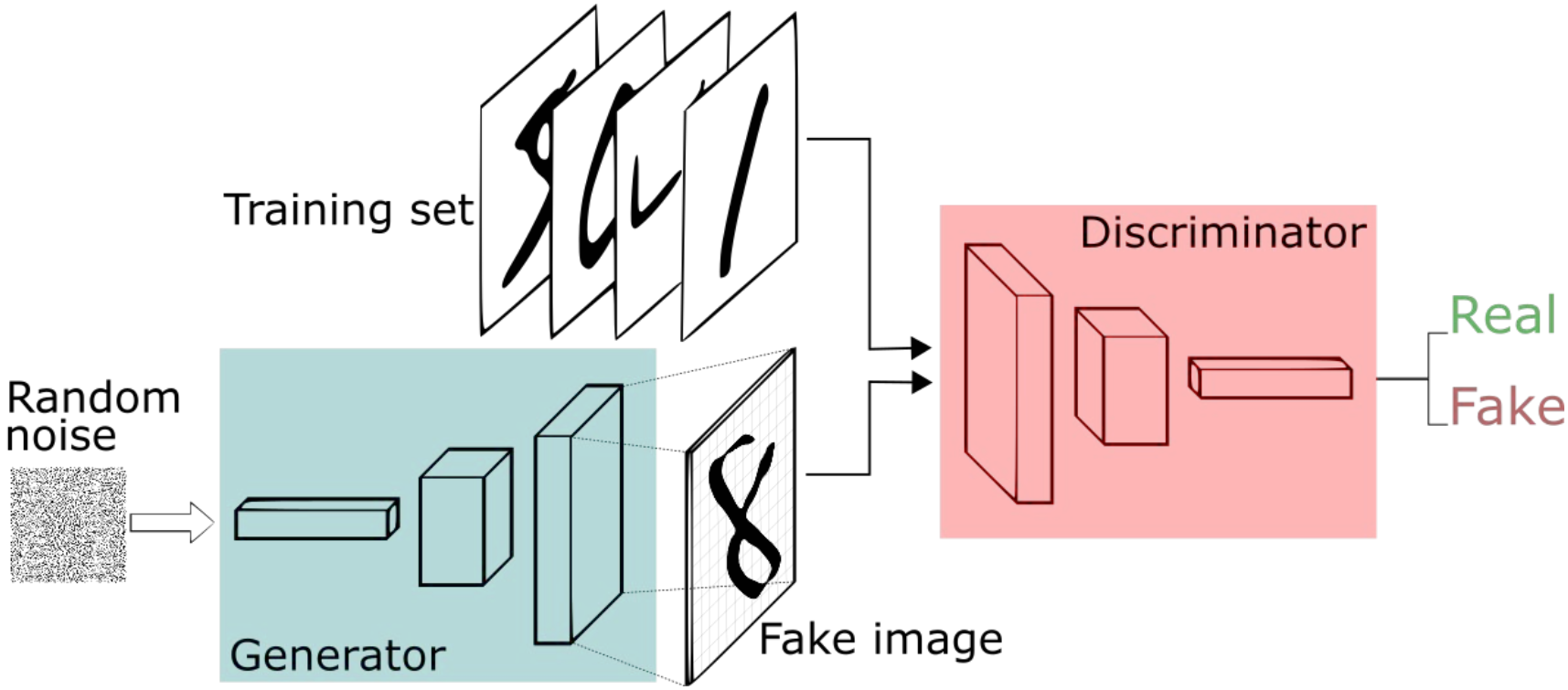


Inverse Reinforcement Learning

- Example

Generative Adversarial Imitation Learning

- GAIL: Generative Adversarial Imitation Learning
- If you are not familiar with GAN:



Generative Adversarial Imitation Learning

- Learn a discriminator to discriminate robot motion from demonstrations
- Use the discriminator as the reward function for RL

Algorithm 1 Generative adversarial imitation learning

1: **Input:** Expert trajectories $\tau_E \sim \pi_E$, initial policy and discriminator parameters θ_0, w_0
2: **for** $i = 0, 1, 2, \ldots$ **do**
3: Sample trajectories $\tau_i \sim \pi_{\theta_i}$
4: Update the discriminator parameters from w_i to w_{i+1} with the gradient

$$\hat{\mathbb{E}}_{\tau_i}[\nabla_w \log(D_w(s,a))] + \hat{\mathbb{E}}_{\tau_E}[\nabla_w \log(1 - D_w(s,a))] \tag{17}$$

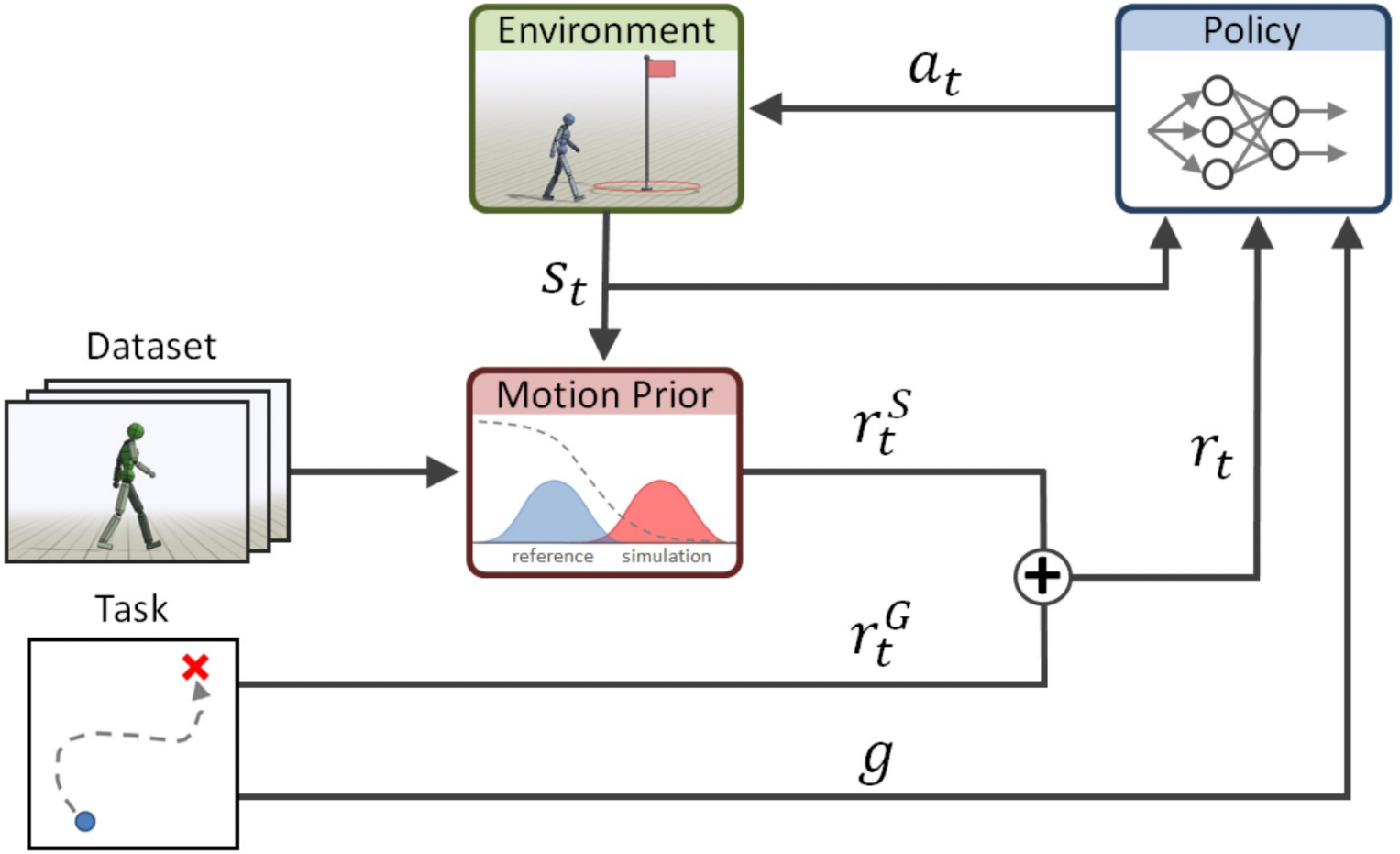
5: Take a policy step from θ_i to θ_{i+1}, using the TRPO rule with cost function $\log(D_{w_{i+1}}(s,a))$. Specifically, take a KL-constrained natural gradient step with

$$\begin{aligned} &\hat{\mathbb{E}}_{\tau_i}[\nabla_\theta \log \pi_\theta(a|s) Q(s,a)] - \lambda \nabla_\theta H(\pi_\theta), \\ &\text{where } Q(\bar{s},\bar{a}) = \hat{\mathbb{E}}_{\tau_i}[\log(D_{w_{i+1}}(s,a)) \mid s_0 = \bar{s}, a_0 = \bar{a}] \end{aligned} \tag{18}$$

6: **end for**

Adversarial Motion Prior

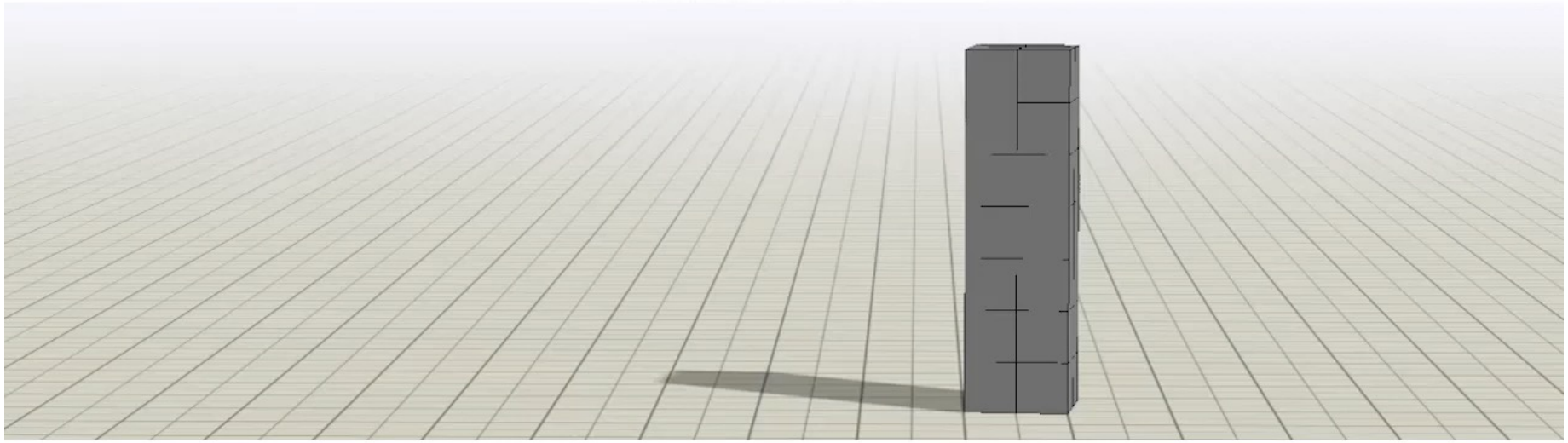
- Is basically GAIL



Adversarial Motion Prior

AMP: Adversarial Motion Priors for Stylized Physics-Based Character Control

(Supplementary Video)



Xue Bin Peng[1], Ze Ma[2], Pieter Abbeel[1], Sergey Levine[1], Angjoo Kanazawa[1]

1 Berkeley UNIVERSITY OF CALIFORNIA

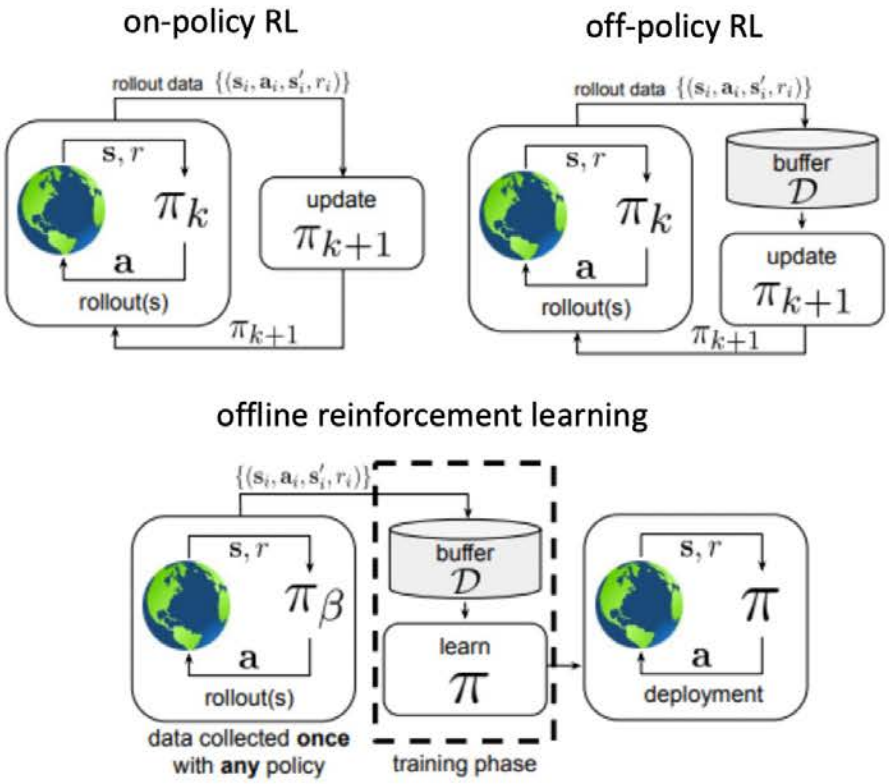
2 上海交通大學 SHANGHAI JIAO TONG UNIVERSITY

Contents

RL with Offline data

- In previous methods
- RL learn from trial and error to maximize rewards
- IL learn by mimicking expert data
- Can we maximize rewards with offline expert data?
  - Do not use simulation, no sim2real gap!
  - Some expert data might be bad, need to learn performance beyond experts
  - Learn from large scale data for scaling

Offline RL



Formally:

$$\mathcal{D} = \{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)\}$$

$$\mathbf{s} \sim d^{\pi_\beta}(\mathbf{s})$$
$$\mathbf{a} \sim \pi_\beta(\mathbf{a}|\mathbf{s})$$ ← generally **not** known
$$\mathbf{s}' \sim p(\mathbf{s}'|\mathbf{s}, \mathbf{a})$$
$$r \leftarrow r(\mathbf{s}, \mathbf{a})$$

RL objective: $\max_\pi \sum_{t=0}^{T} E_{\mathbf{s}_t \sim d^\pi(\mathbf{s}), \mathbf{a}_t \sim \pi(\mathbf{a}|\mathbf{s})}[\gamma^t r(\mathbf{s}_t, \mathbf{a}_t)]$

5

Offline RL

- Offline RL can perform better than the dataset

Supervised Learning

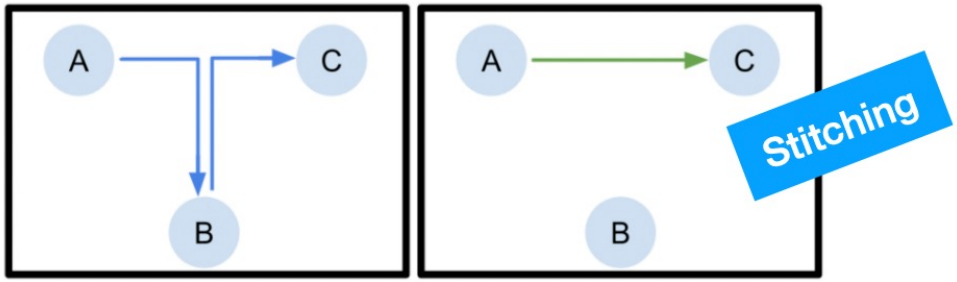
Can do as good as the dataset!

→ Dog

→ Cat?

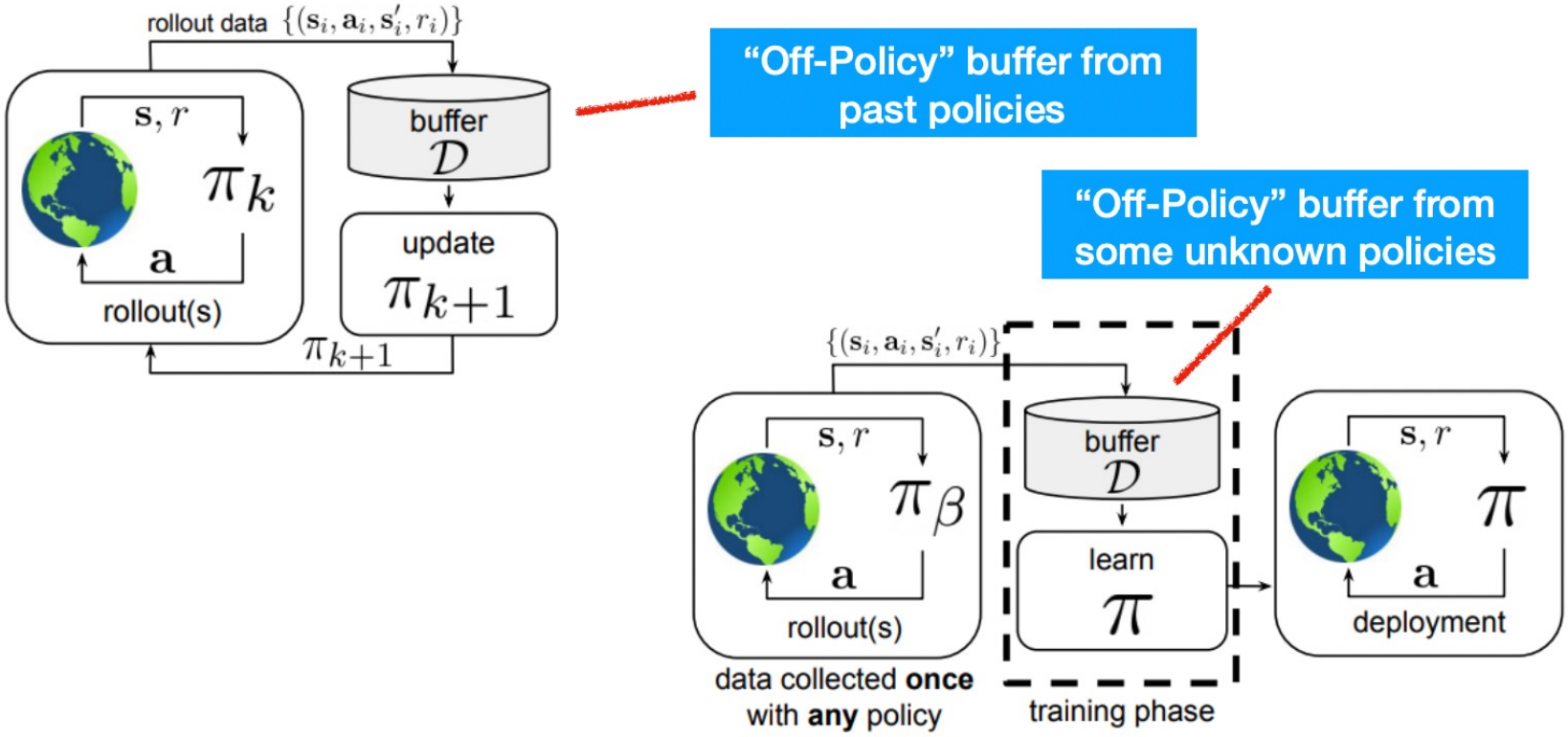
Offline Reinforcement Learning

Can do **better** than the dataset!



Off-policy RL

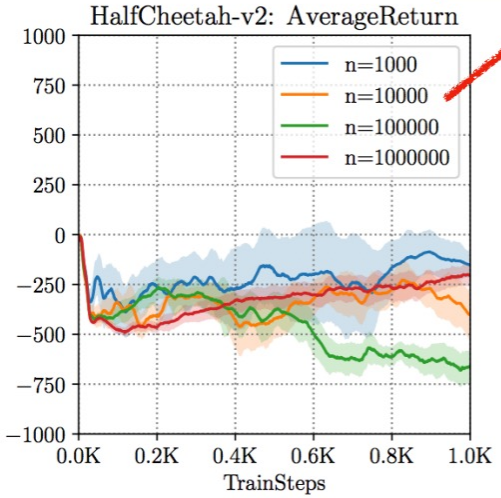
- Can we directly use a variant of off-policy RL?



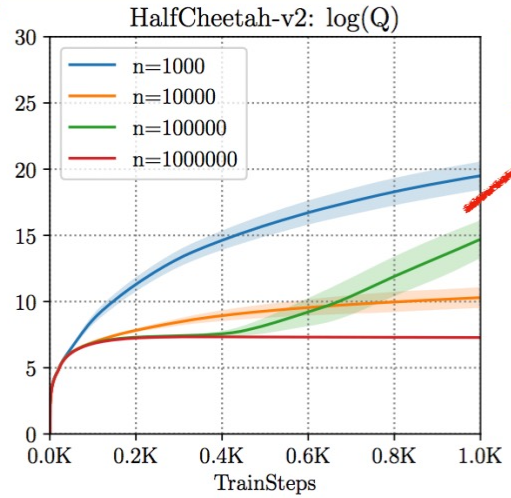
Problems of Off-policy RL

- If we collect data and just run off-policy RL on this data

Performance doesn't improve with more data



how well it does



how well it thinks it does

Learning diverges

"Policy unlearning"

Not a classical overfitting issue!

- Why?

Problems of Off-policy RL

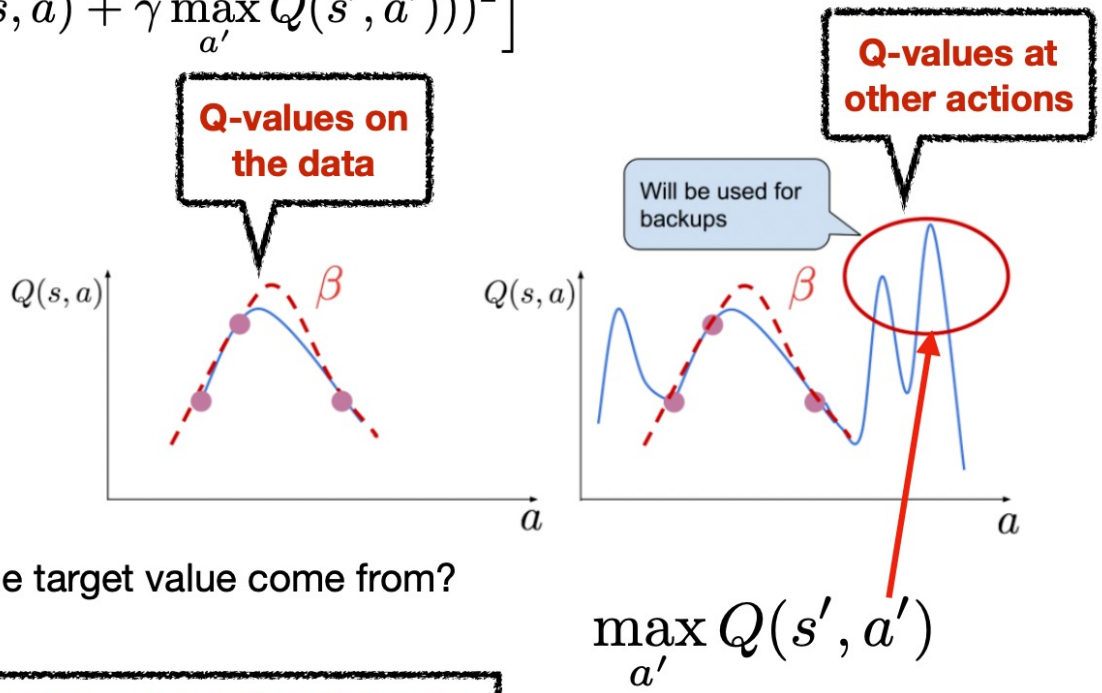
- Let's see how the Q-function is updated

$$Q(s,a) \leftarrow r(s,a) + \gamma \max_{a'} Q(s',a')$$

$$\mathbb{E}_{s,a,s' \sim \mathcal{D}} \left[\left(Q(s,a) - \left(r(s,a) + \gamma \max_{a'} Q(s',a') \right) \right)^2 \right]$$

Which actions does the Q-function train on?

$$s, a \sim \mathcal{D}$$



Where does the action a' for the target value come from?

Q-learning queries values at unseen action targets, which are never trained during training

Distributional Shift in Offline RL

- Distribution shift between the behavior policy (the policy that collected the data) and the policy during learning

$$Q(s,a) \leftarrow r(s,a) + \gamma \max_{a'} Q(s',a')$$

$\neq \pi_\beta(a|s)$

$$Q(s,a) \leftarrow r(s,a) + \gamma \mathbb{E}_{a' \sim \pi(a'|s')} Q(s',a')$$

$= \pi_\beta(a|s)$

Training: $\mathbb{E}_{s,a \sim d^{\pi_\beta}(s,a)} \left[(Q(s,a) - \mathcal{B}\bar{Q}(s,a))^2 \right]$

Offline Q-Learning algorithms can overestimate the value of unseen actions and can thus be **falsely optimistic**

Distributional Shift in Offline RL

- This phenomenon also happens in online RL settings, where the Q-function is erroneously optimistic
- However, in online settings, the behavior policy is updating with exploration for new actions
- But this is impossible in offline RL, due to no access to an environment

Adding Policy Constraint

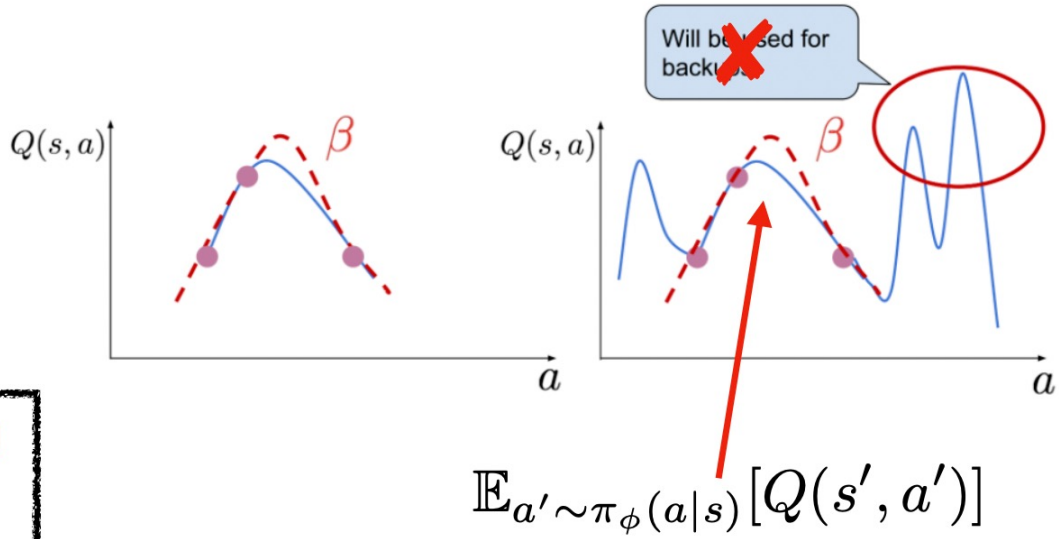
- One way to address the distributional shift is to add a policy constraint

$$Q(s,a) \leftarrow r(s,a) + \gamma \mathbb{E}_{a' \sim \pi_\phi(a|s)}[Q(s',a')]$$

"Policy Constraint"

$$\pi_\phi := \arg\max_\phi E_{a \sim \pi_\phi(a|s)}[Q(s,a)] \quad \text{s.t.} \quad D(\pi_\phi(a|s), \pi_\beta(a|s)) \leq \varepsilon$$

Out-of-distribution action values are no longer used for the backup



Hence, all values used during training are also trained, leading to better learning

Adding Policy Constraint

- There are different types of policy constraint we can add, leading to different offline RL algorithms

$$\pi_\phi := \arg\max_\phi E_{a\sim\pi_\phi(a|s)}[Q(s,a)] \quad \text{s.t.} \quad D(\pi_\phi(a|s), \pi_\beta(a|s)) \le \varepsilon$$

Several Ways of Implementing Them:

$$D(\pi_\phi, \pi_\beta) = \mathrm{MMD}(\pi_\phi, \pi_\beta)$$

- Support matching (**Kumar** et al. 2019, Laroche et al. 2019, Wu et al. 2019)
- Distribution matching (Peng et al. 2019, Fujimoto et al. 2019, Jaques et al. 2019)

$$D(\pi_\phi, \pi_\beta) = \mathrm{D_{KL}}(\pi_\phi, \pi_\beta)$$

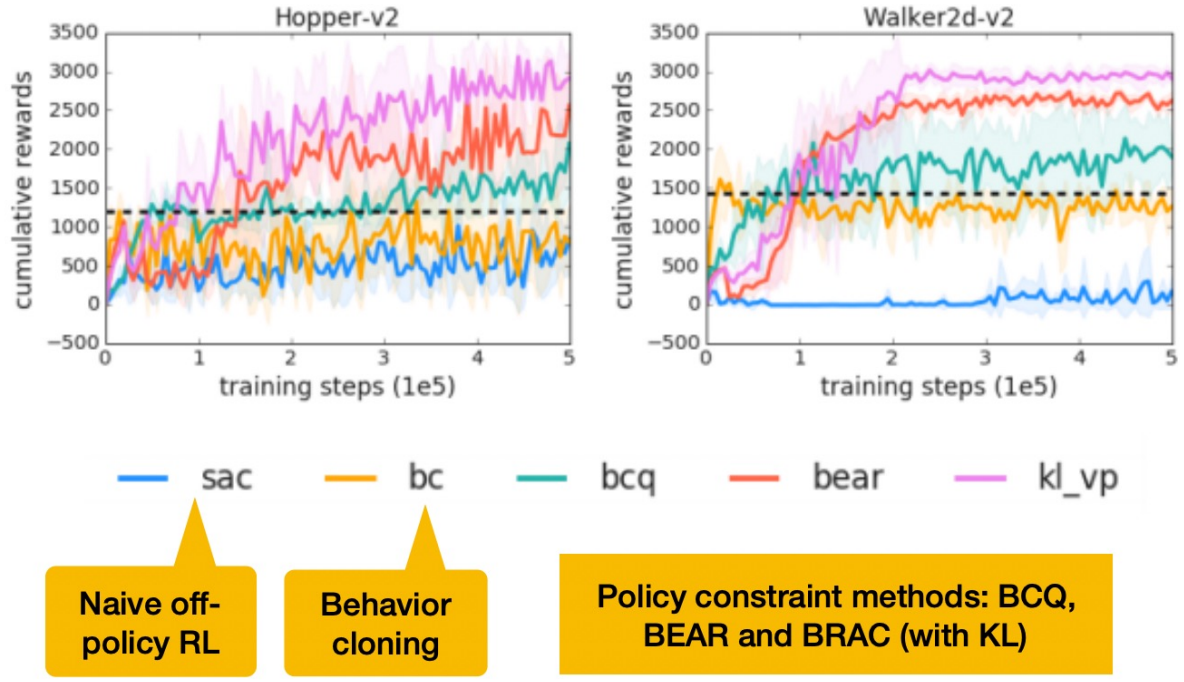
- State-marginal constraints (Nachum & Dai 2020)

$$D(\pi_\phi, \pi_\beta) = D(d^{\pi_\phi}(s,a), d^{\pi_\beta}(s,a))$$

- Implicit /closed-form distribution constraints (Peng et al. 2019, Nair et al. 2020, Wang et al. 2020)

Adding Policy Constraint

- Show better performance than behavior cloning



Domain	Task Name	BC	SAC	BEAR	BRAC-p	BRAC-v
AntMaze	antmaze-umaze	65.0	0.0	**73.0**	50.0	70.0
	antmaze-umaze-diverse	55.0	0.0	61.0	40.0	70.0
	antmaze-medium-play	0.0	0.0	0.0	0.0	0.0
	antmaze-medium-diverse	0.0	0.0	8.0	0.0	0.0
	antmaze-large-play	0.0	0.0	0.0	0.0	0.0
	antmaze-large-diverse	0.0	0.0	0.0	0.0	0.0

Adding Policy Constraint

- Are policy constraint methods sufficient?
- Require estimation of the behavior policy
  - If the behavior policy is wrongly estimated, the policy constraint can fail

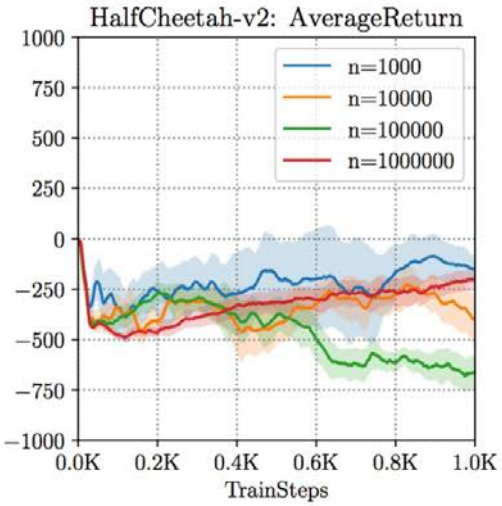
$$\pi_\phi := \arg\max_\phi E_{a\sim\pi_\phi(a|s)}[Q(s,a)] \quad \text{s.t.} \quad D(\pi_\phi(a|s), \pi_\beta(a|s)) \leq \varepsilon$$

estimated from data

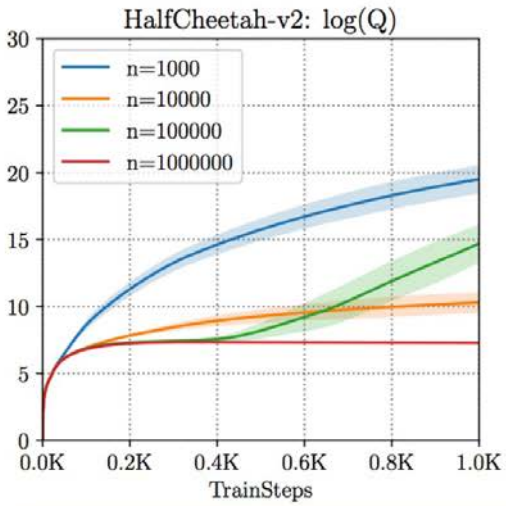
- Often tend to be too conservative

Tackle Over-estimation

- Let's revisit the problem



how well it does



how well it thinks it does

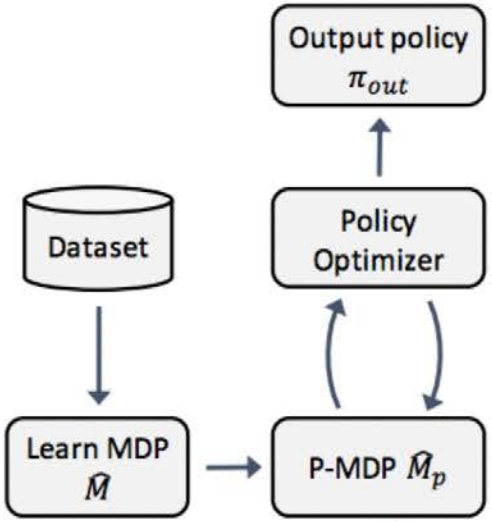
Can we directly tackle false over-estimation, instead of fixes to avoid out-of-distribution actions?

In some cases, not all out-of-distribution actions are bad, they are bad if they affect the policy (i.e. when values are overestimated)

Can we devise methods that learn lower-bounds on the policy value/ performance?

Yes! Two ways: model-based **and** model-free

Conservative Model-based RL

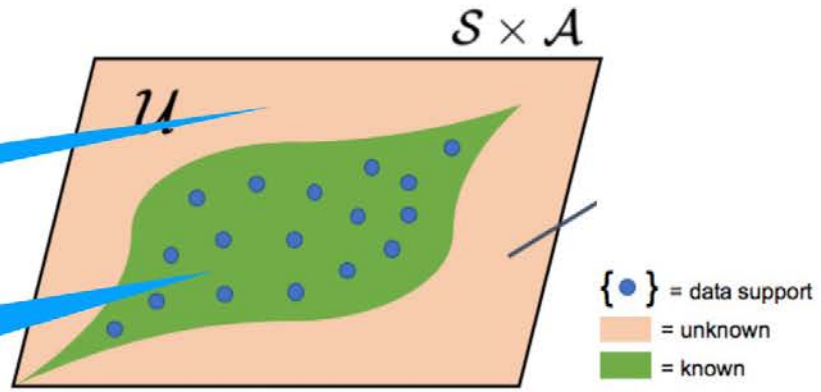


1. Learn a dynamic model P(s'|s, a) from the offline data.
2. Learn a conservative/ "pessimistic" estimate of the reward function.
3. Perform policy optimisation (e.g., via planning or Dyna) with the learned model and the reward function.

This is the new bit!

Make rewards pessimistic

Keep unaltered reward



Conservative Model-based RL

$$\tilde{r}(s,a) = r(s,a) - \lambda u(s,a).$$

MOPO (Yu et al. 2020)

Covariance matrix of an ensemble of dynamics models

$$\tilde{r}_j = r_j - \lambda \max_{i=1}^{N} \|\Sigma^i(s_j, a_j)\|_{\mathrm{F}}$$

MBPO (Dyna)

MOReL (Kidambi et al. 2020)

Disagreement in an ensemble of dynamics models

$$\mathrm{disc}(s,a) = \max_{i,j} \left\| f_{\phi_i}(s,a) - f_{\phi_j}(s,a) \right\|_2$$

Planning

$$\tilde{r}(s,a) = -R_{\max} \quad \text{if } \mathrm{disc}(s,a) > \text{threshold}$$

Conservative Model-based RL

- Better than policy constraint methods generally

Dataset type	Environment	BC	MOPO (ours)	MBPO	SAC	BEAR	BRAC-v
random	halfcheetah	2.1	**31.9** $\pm$ 2.8	30.7 $\pm$ 3.9	30.5	25.5	28.1
random	hopper	1.6	**13.3** $\pm$ 1.6	4.5 $\pm$ 6.0	11.3	9.5	12.0
random	walker2d	9.8	**13.0** $\pm$ 2.6	8.6 $\pm$ 8.1	4.1	6.7	0.5
medium	halfcheetah	36.1	40.2 $\pm$ 2.7	28.3 $\pm$ 22.7	-4.3	38.6	**45.5**
medium	hopper	29.0	26.5 $\pm$ 3.7	4.9 $\pm$ 3.3	0.8	**47.6**	32.3
medium	walker2d	6.6	14.0 $\pm$ 10.1	12.7 $\pm$ 7.6	0.9	33.2	**81.3**
mixed	halfcheetah	38.4	**54.0** $\pm$ 2.6	47.3 $\pm$ 12.6	-2.4	36.2	45.9
mixed	hopper	11.8	**92.5** $\pm$ 6.3	49.8 $\pm$ 30.4	1.9	10.8	0.9
mixed	walker2d	11.3	**42.7** $\pm$ 8.3	22.2 $\pm$ 12.7	3.5	25.3	0.8
med-expert	halfcheetah	35.8	**57.9** $\pm$ 24.8	9.7 $\pm$ 9.5	1.8	51.7	45.3
med-expert	hopper	111.9	51.7 $\pm$ 42.9	**56.0** $\pm$ 34.5	1.6	4.0	0.8
med-expert	walker2d	6.4	55.0 $\pm$ 19.1	7.6 $\pm$ 3.7	-0.1	26.0	**66.6**

Learning Lower-Bounded Q-values

- Conservative Q-Learning (CQL) Algorithm
- Since learned Q-values (our belief of policy values) are overestimated, let's make them provably lower bound the true value

Minimize big Q-values

Standard Bellman Error

$$\hat{Q}^{\pi}_{\text{CQL}} := \min_{Q} \max_{\mu} \mathbb{E}_{a\sim\mu(a|s)}[Q(s,a)] + \frac{1}{2\alpha}\mathbb{E}_{s,a,s'\sim\mathcal{D}}\left[(Q(s,a) - (r(s,a) + \gamma\mathbb{E}_{a\sim\pi_\phi(a|s)}[\bar{Q}(s',a')]))^2\right]$$

$$\hat{Q}^{\pi}_{\text{CQL}}(s,a) \leq Q(s,a) \quad \forall s \in \mathcal{D}, a$$

CQL-v1

CQL Algorithm:

1. Learn $\hat{Q}^{\pi}_{\text{CQL}}$ using offline data $\mathcal{D}$.
2. Optimize policy w.r.t. $\hat{Q}^{\pi}_{\text{CQL}}$: $\pi \leftarrow \arg\max_{\pi} \mathbb{E}_{\pi}[\hat{Q}^{\pi}_{\text{CQL}}]$.

Learning Lower-Bounded Q-values

- Practical CQL algorithm

$$\min_Q \alpha \mathbb{E}_{\mathbf{s}\sim\mathcal{D}}\left[\log \sum_{\mathbf{a}} \exp(Q(\mathbf{s},\mathbf{a})) - \mathbb{E}_{\mathbf{a}\sim\hat{\pi}_\beta(\mathbf{a}|\mathbf{s})}\left[Q(\mathbf{s},\mathbf{a})\right]\right] + \frac{1}{2}\mathbb{E}_{\mathbf{s},\mathbf{a},\mathbf{s}'\sim\mathcal{D}}\left[\left(Q - \hat{\mathcal{B}}^{\pi_k}\hat{Q}^k\right)^2\right].$$

Algorithm 1 Conservative Q-Learning (both variants)

1: Initialize Q-function, Q_θ, and optionally a policy, π_ϕ.
2: **for** step t in $\{1, \ldots, \mathrm{N}\}$ **do**
3: Train the Q-function using G_Q gradient steps on objective from Equation 4
$\theta_t := \theta_{t-1} - \eta_Q \nabla_\theta \mathrm{CQL}(\mathcal{R})(\theta)$
(Use $\mathcal{B}^*$ for Q-learning, $\mathcal{B}^{\pi_{\phi_t}}$ for actor-critic)
4: (only with actor-critic) Improve policy π_ϕ via G_π gradient steps on ϕ with SAC-style entropy regularization:
$\phi_t := \phi_{t-1} + \eta_\pi \mathbb{E}_{\mathbf{s}\sim\mathcal{D},\mathbf{a}\sim\pi_\phi(\cdot|\mathbf{s})}[Q_\theta(\mathbf{s},\mathbf{a}) - \log \pi_\phi(\mathbf{a}|\mathbf{s})]$
5: **end for**

Only change on top of standard Deep Q-Learning

Contents

The exploration problem

this is easy (mostly)

Why?

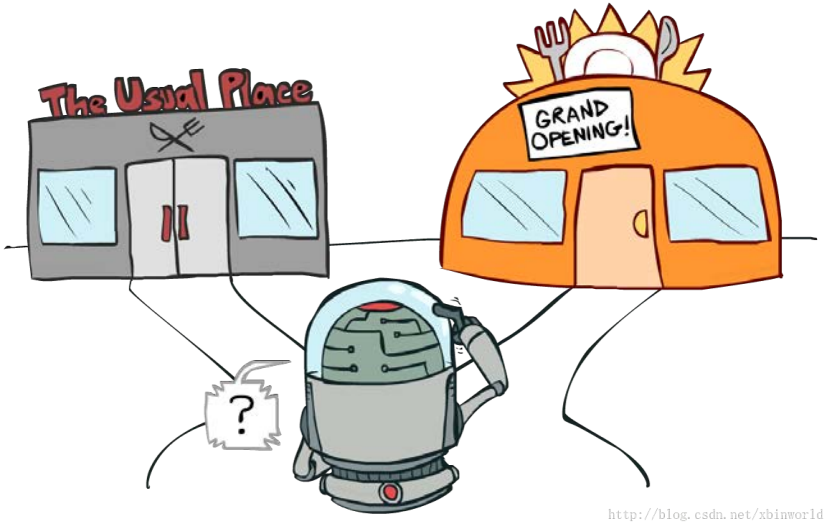
this is impossible

The exploration problem

- Montezuma's revenge
  - Getting key = reward
  - Opening door = reward
  - Getting killed by skull = nothing (is it good? bad?)
  - Finishing the game only weakly correlates with rewarding events
  - We know what to do because we understand what these sprites mean!

Exploration and Exploitation

- **Exploration** is equally important to **Exploitation**



Go to your favorite restaurant?

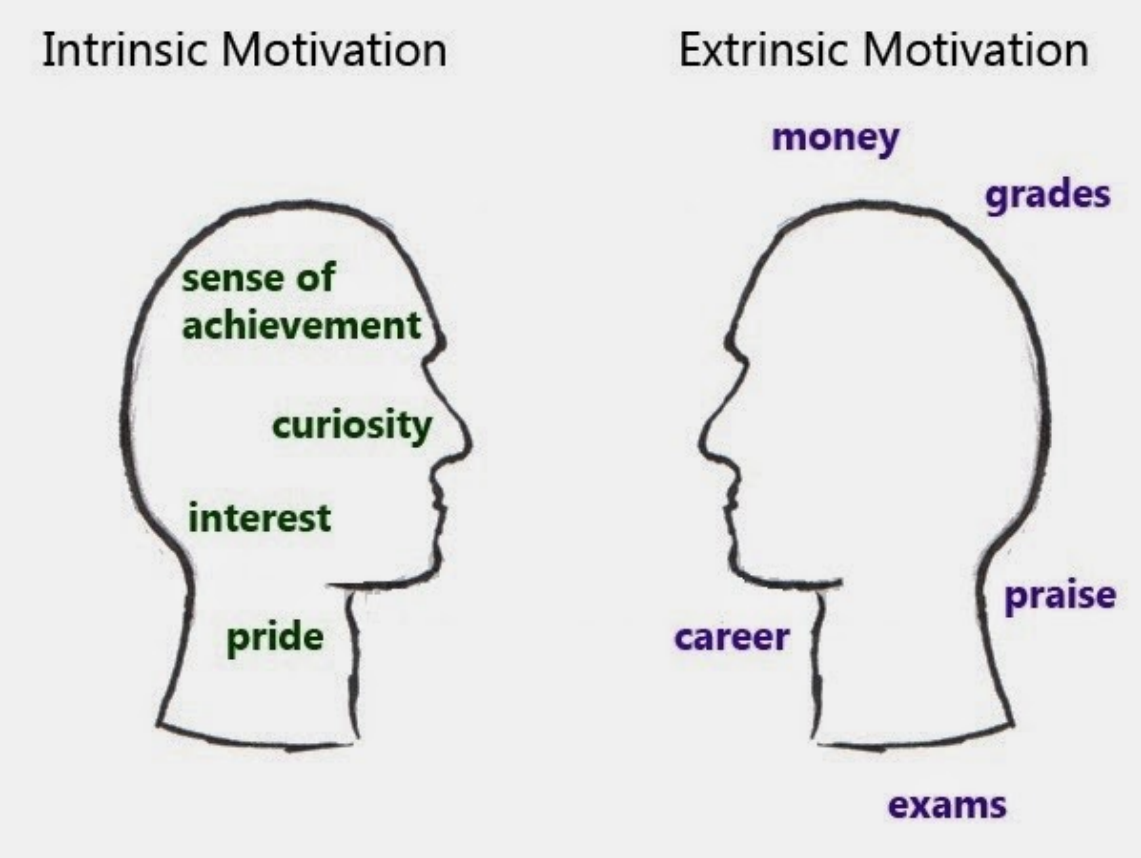
Try a new restaurant?

Handle exploration

- How to handle exploration problem in deep RL
  - Intrinsic reward
  - Unsupervised RL

Intrinsic Reward

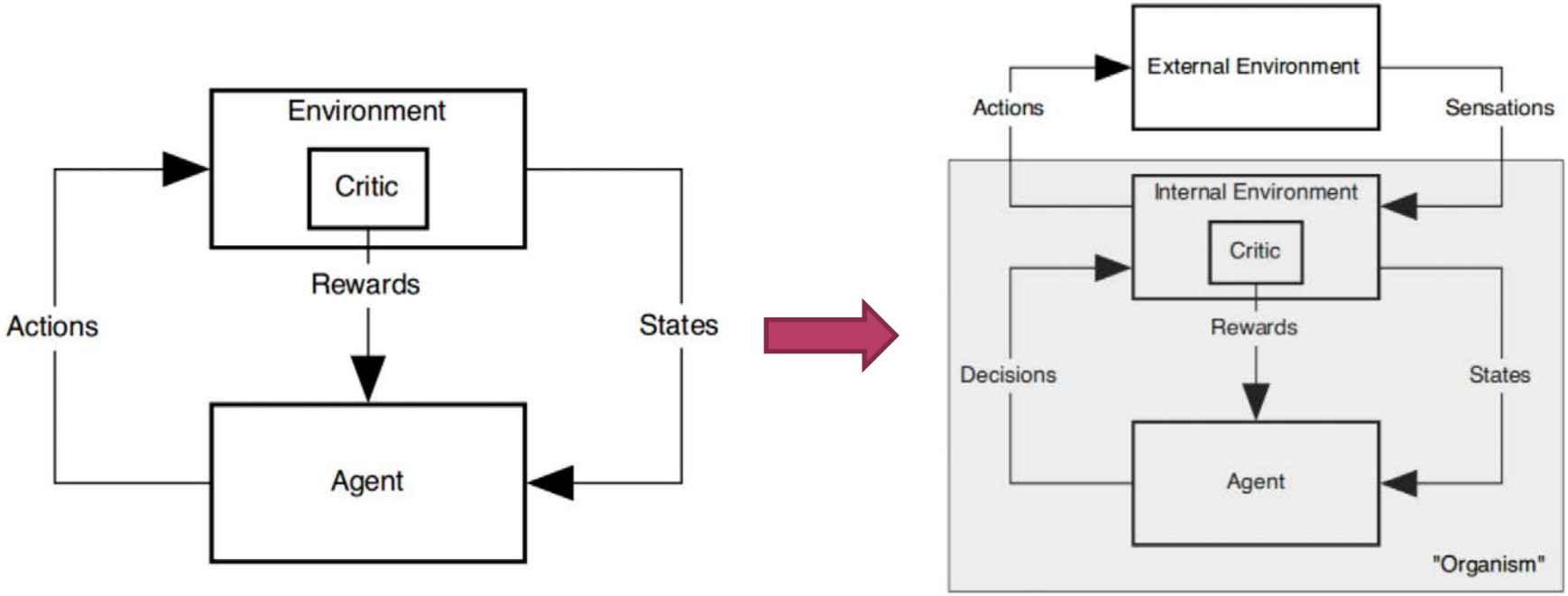
- Intrinsic reward vs extrinsic reward



- Intrinsic reward can help optimize extrinsic reward

Intrinsic Reward

- Intrinsic reward in reinforcement learning



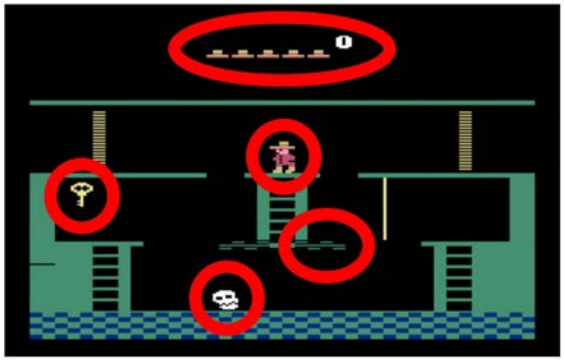
Count-based Intrinsic Reward

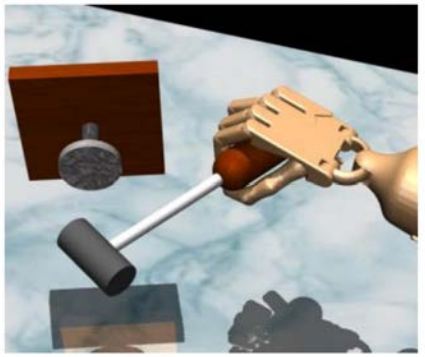
- Key idea: encourage the agent to explore states that have not been seen
- The reward can be modified to:

$$r^{+}(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \mathcal{B}(N(\mathbf{s}))$$

bonus that decreases with $N(\mathbf{s})$

- Here $N(s)$ is the times state s has been visited
- But how to count state, in continuous space?





Prediction Error Based Intrinsic Reward

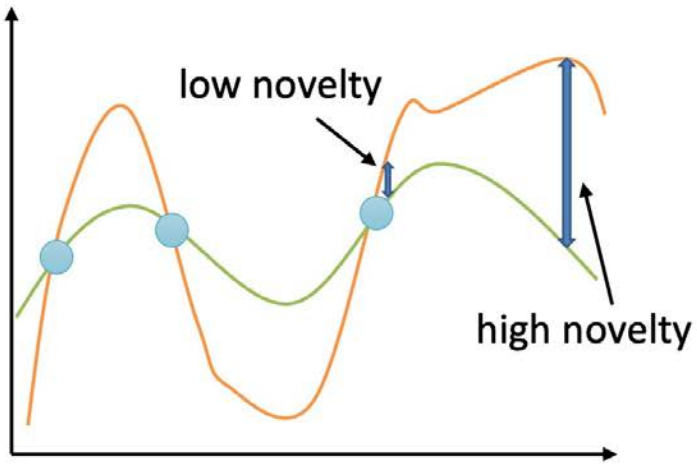
- Instead of encouraging exploration based on counting, we can directly encourage when a state is novel
- How to calculate the novelty of a state?
- Idea: use prediction error

let's say we have some **target** function $f^\star(\mathbf{s}, \mathbf{a})$

given our buffer $\mathcal{D} = \{(\mathbf{s}_i, \mathbf{a}_i)\}$, fit $\hat{f}_\theta(\mathbf{s}, \mathbf{a})$

use $\mathcal{E}(\mathbf{s}, \mathbf{a}) = ||\hat{f}_\theta(\mathbf{s}, \mathbf{a}) - f^\star(\mathbf{s}, \mathbf{a})||^2$ as bonus

$$f^\star(\mathbf{s}, \mathbf{a}) = \mathbf{s}'$$



Information Theoretic Intrinsic Reward

- Key idea
  - Determine some variable z
  - Estimate the variable's entropy gain, when given some new information
  - We want to get more information for exploration

let $\mathcal{H}(\hat{p}(z))$ be the current entropy of our z estimate

let $\mathcal{H}(\hat{p}(z)|y)$ be the entropy of our z estimate after observation y

the lower the entropy, the more precisely we know z

$$\mathrm{IG}(z, y) = E_y[\mathcal{H}(\hat{p}(z)) - \mathcal{H}(\hat{p}(z)|y)]$$

Information Theoretic Intrinsic Reward

- What z to learn? We can learn about transitions:

$$p_\theta(s_{t+1}|s_t, a_t): \; z = \theta$$

- Intuition: a transition is more informative if it causes the the learned dynamics model to change more
- Information gain can be equivalently written as:

$$D_{\mathrm{KL}}(p(z|y)\|p(z))$$

- Thus it can be equivalently written as:

$$D_{\mathrm{KL}}(p(\theta|h, s_t, a_t, s_{t+1})\|p(\theta|h))$$

model parameters for $p_\theta(s_{t+1}|s_t, a_t)$

history of all prior transitions

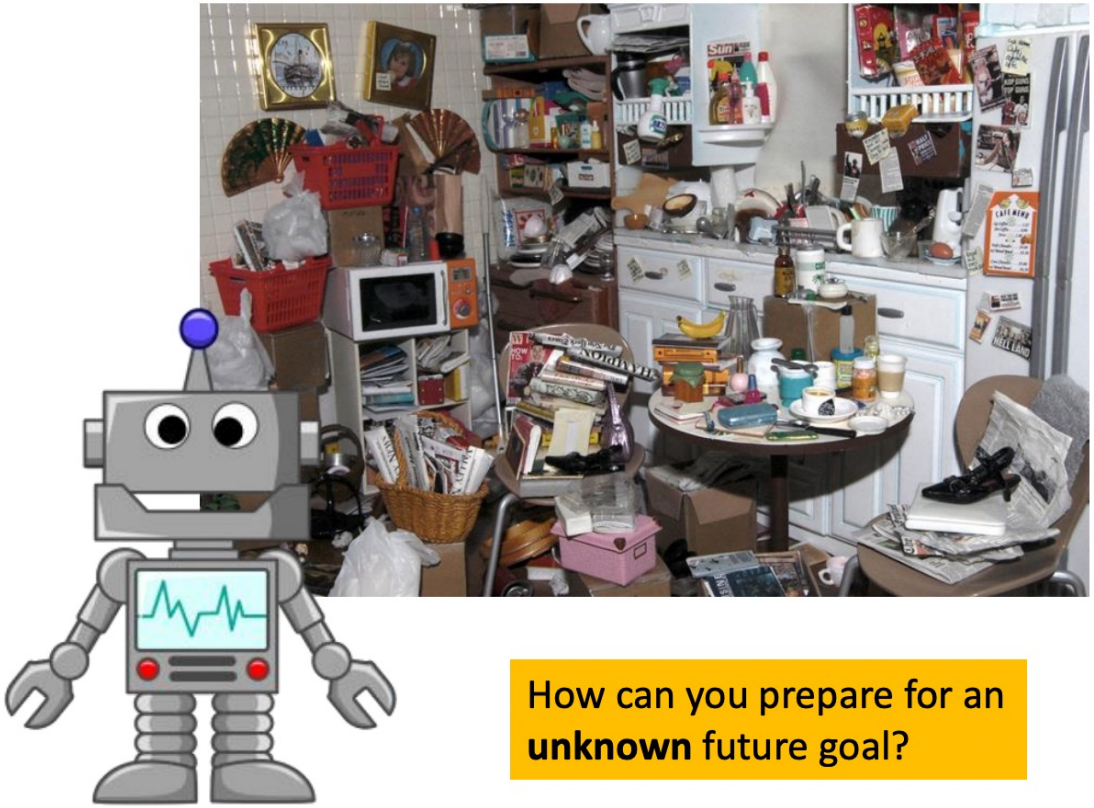
newly observed transition

Unsupervised learning of diverse behaviors

- Human usually learn without any reward function at all

Unsupervised learning of diverse behaviors

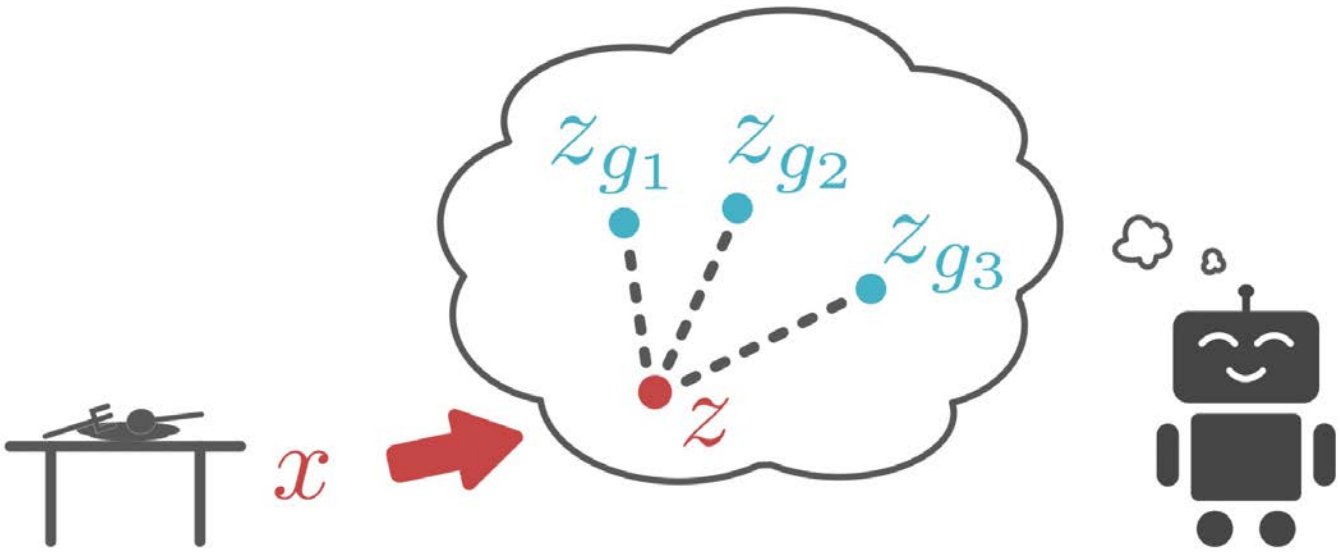
- Explore the space of possible behaviors
- Learn skills without supervision, then use them to accomplish goals



How can you prepare for an **unknown** future goal?

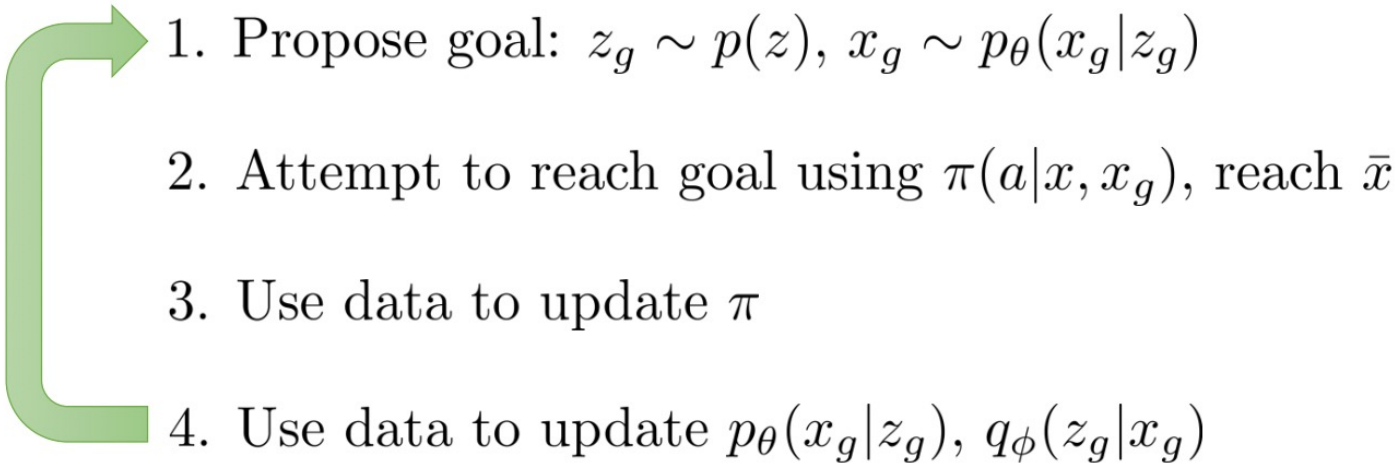
Goal-conditioned RL

- Self propose goals, and then learn to reach the goal



Goal-conditioned RL

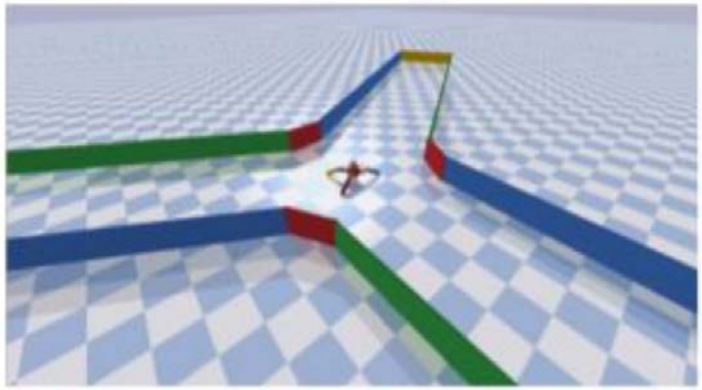
- Self propose goals, and then learn to reach the goal



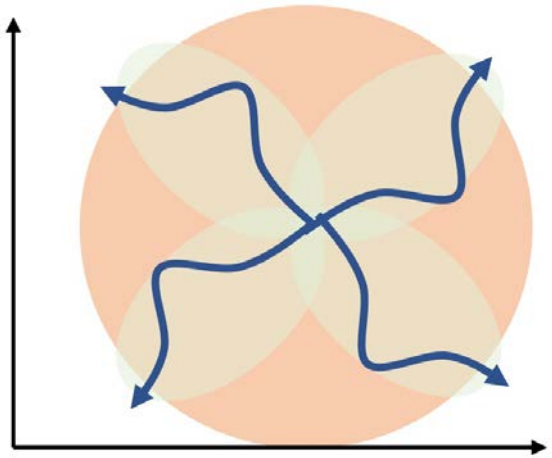
State distribution matching

learn $\pi(\mathbf{a}|\mathbf{s})$ so as to minimze $D_{\mathrm{KL}}(p_\pi(\mathbf{s})\|p^\star(\mathbf{s}))$

target state density



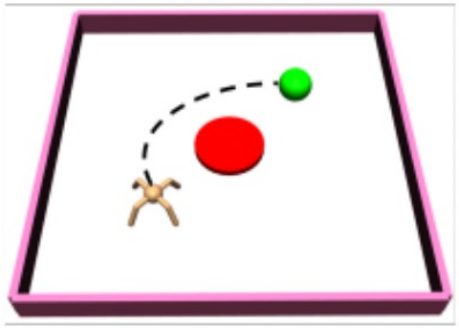
much better coverage!

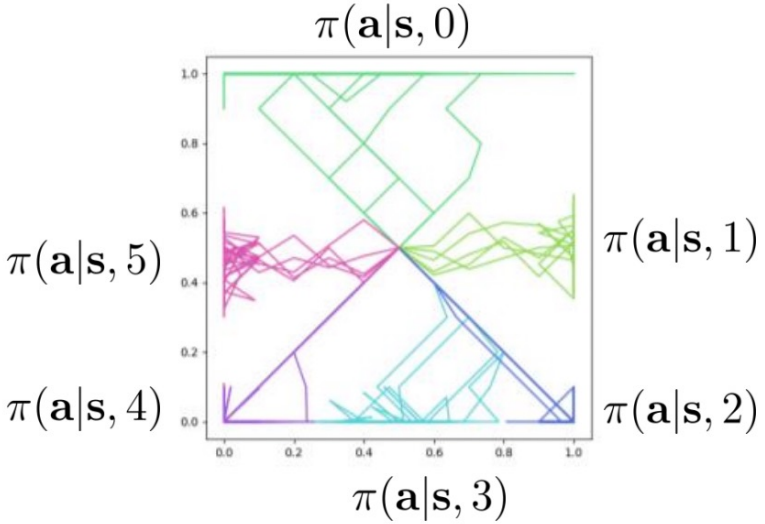


Skill distribution matching

$$\pi(\mathbf{a}|\mathbf{s}, z)$$

task index





Contents

The challenge we face

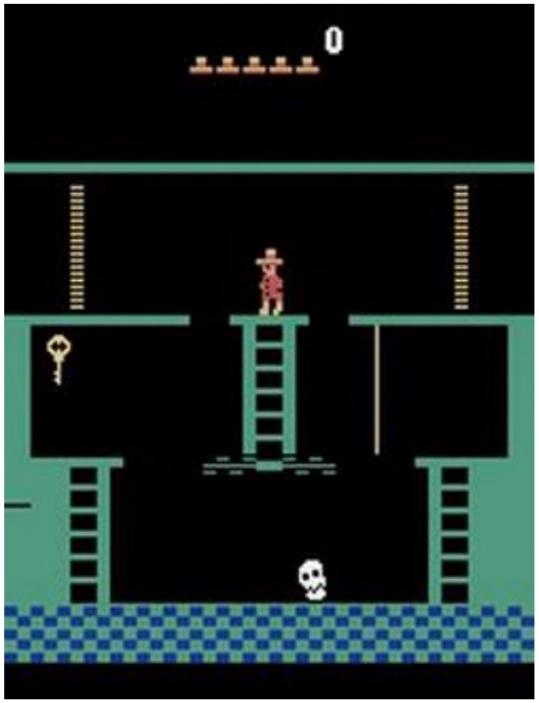
this is easy (mostly)

Why?

this is impossible

The challenge we face

- Montezuma's revenge
  - We know what to do because we understand what these sprites mean!
  - Key: we know it opens doors!
  - Ladders: we know we can climb them!
  - Skull: we don't know what it does, but we know it can't be good!
  - Prior understanding of problem structure can help us solve complex tasks quickly!



Transfer learning

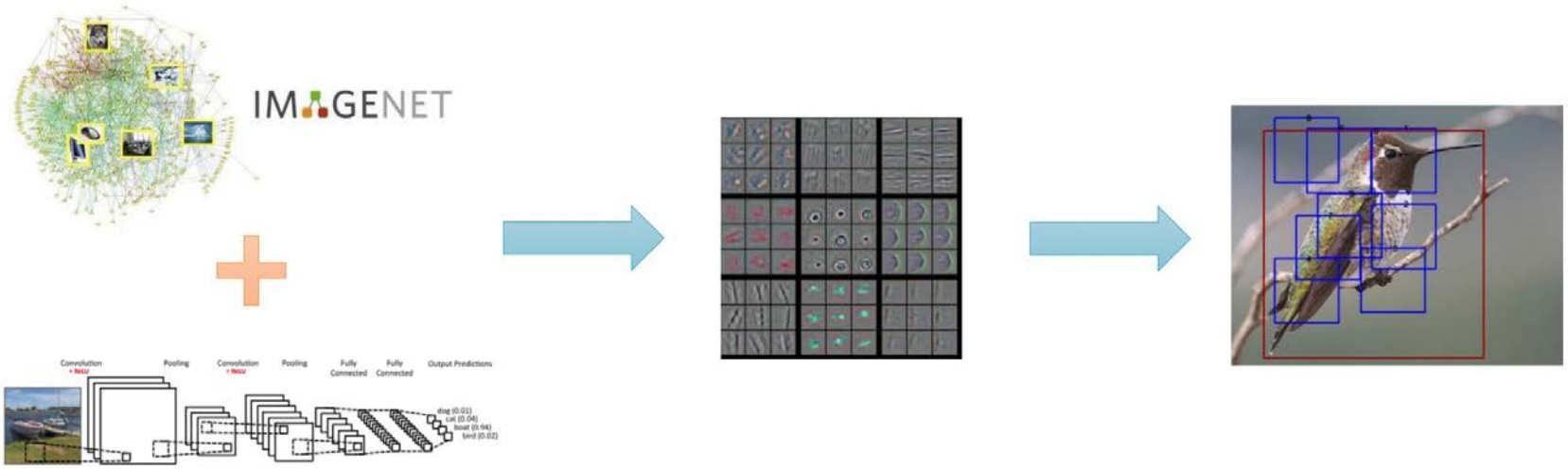
- Using experience from one set of tasks for faster learning and better performance on a new task

source domain

target domain

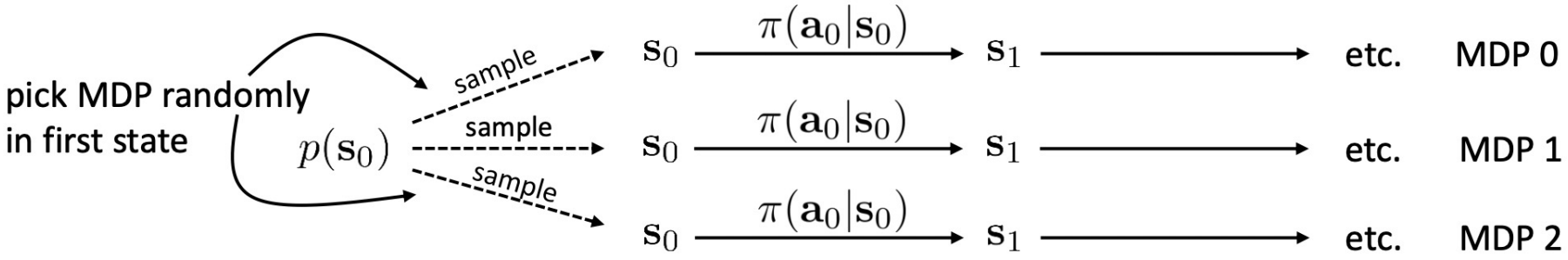
Transfer learning

- We usually use the most popular transfer learning method in (supervised) deep learning!



Multi-task learning

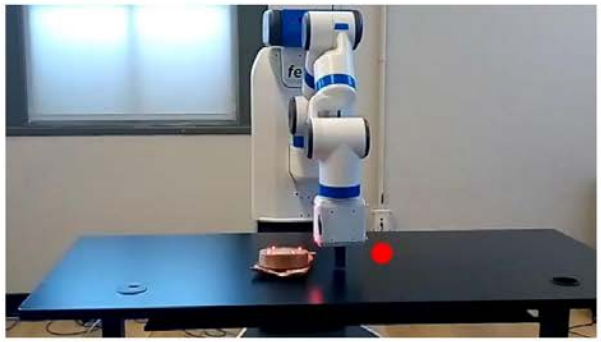
- Pretrain in multiple tasks, then finetune in target task
  - Zero-shot (No finetune needed!)
  - Few-shot



Multi-task learning

- Zero-shot: Domain randomization

Sadeghi et al., "CAD2RL: Real Single-Image Flight without a Single Real Image." 2016

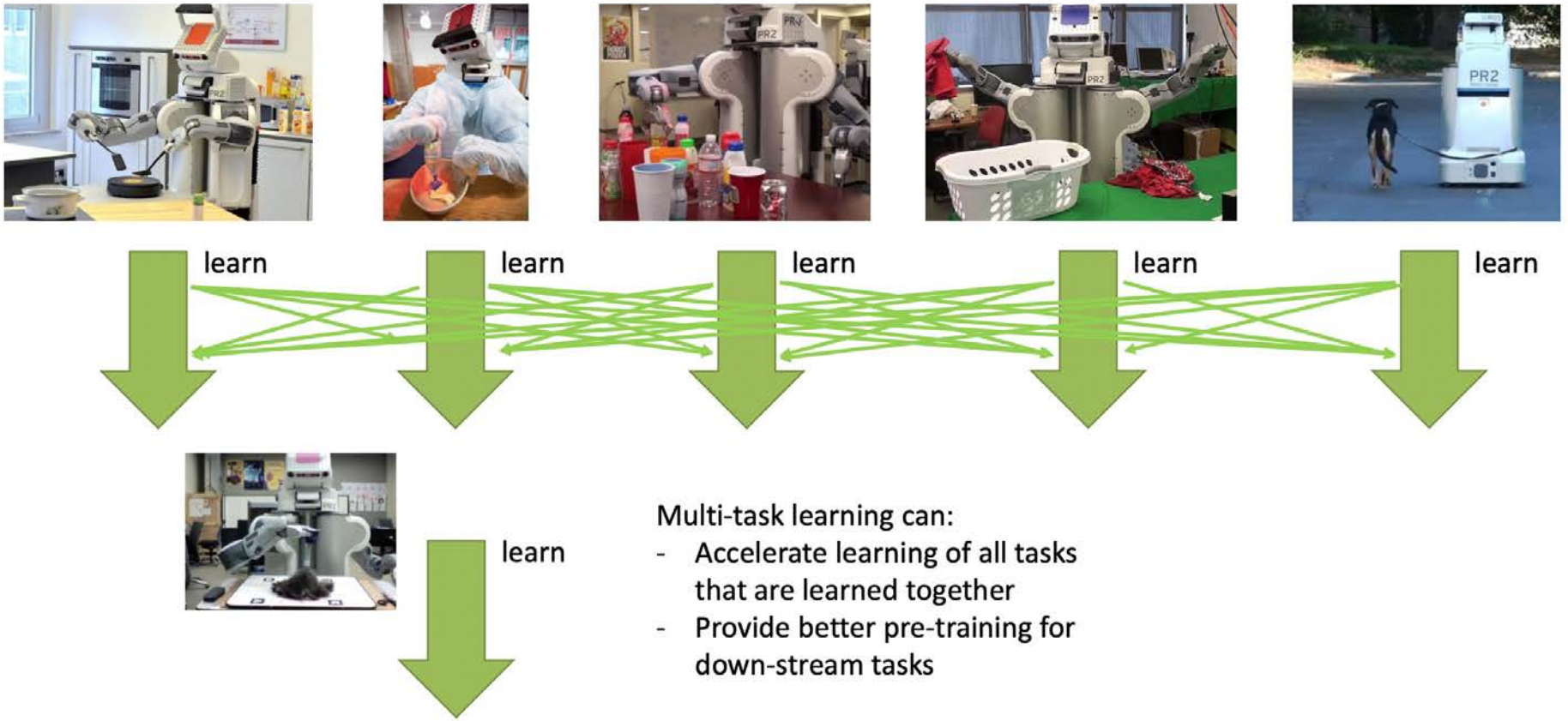


Xue Bin Peng et al., "Sim-to-Real Transfer of Robotic Control with Dynamics Randomization." 2018

Lee et al., "Learning Quadrupedal Locomotion over Challenging Terrain." 2020

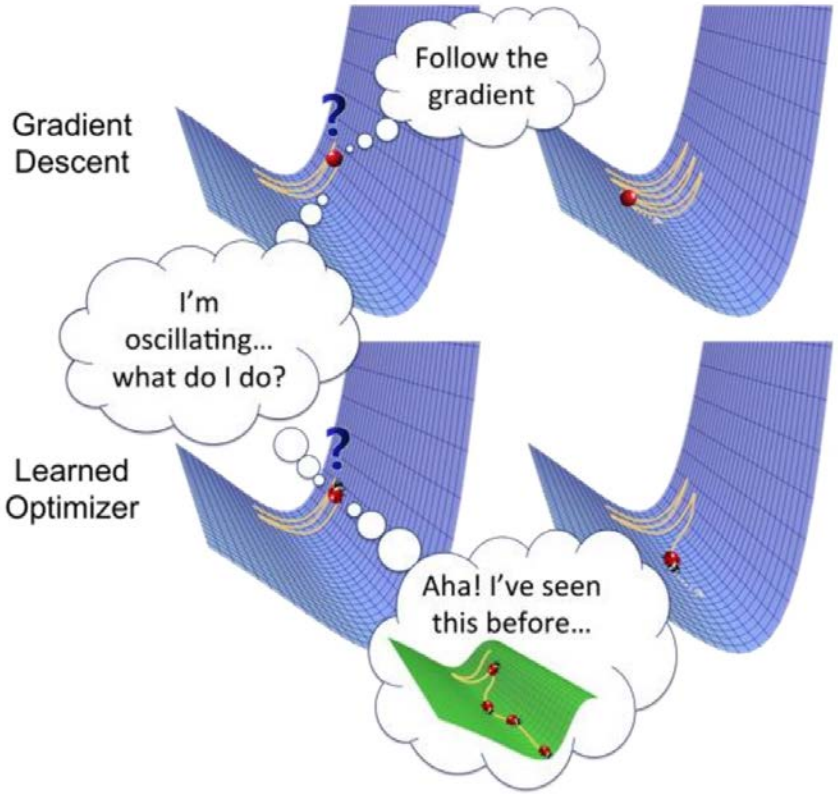
Multi-task learning

- Few-shot: fine-tuning on target task



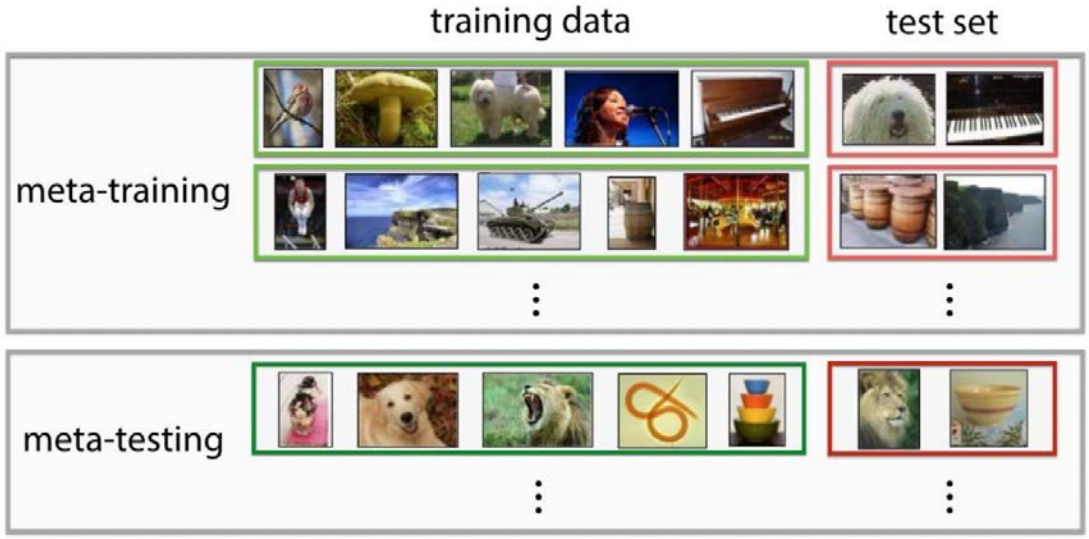
Meta-learning

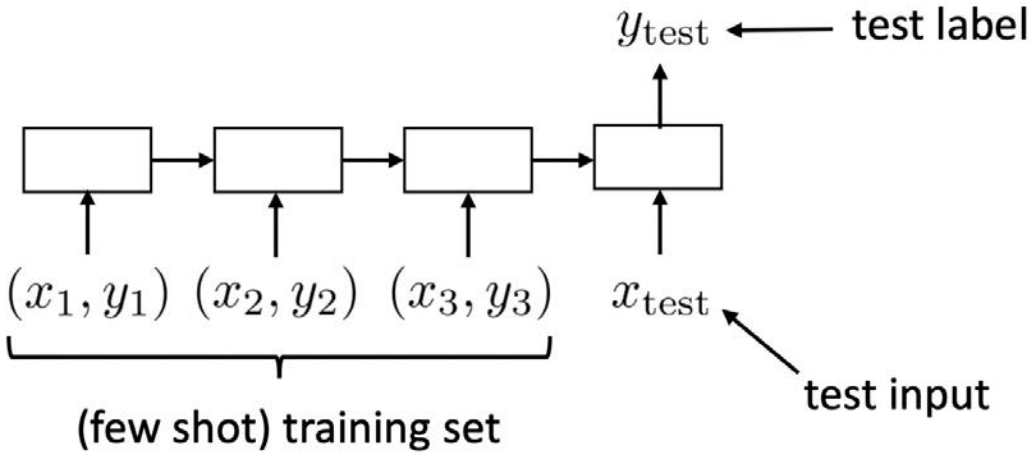
- Meta-learning=Learning to learn!
- If we can meta-learn a faster reinforcement learner, we can learn new tasks efficiently!



Meta-learning

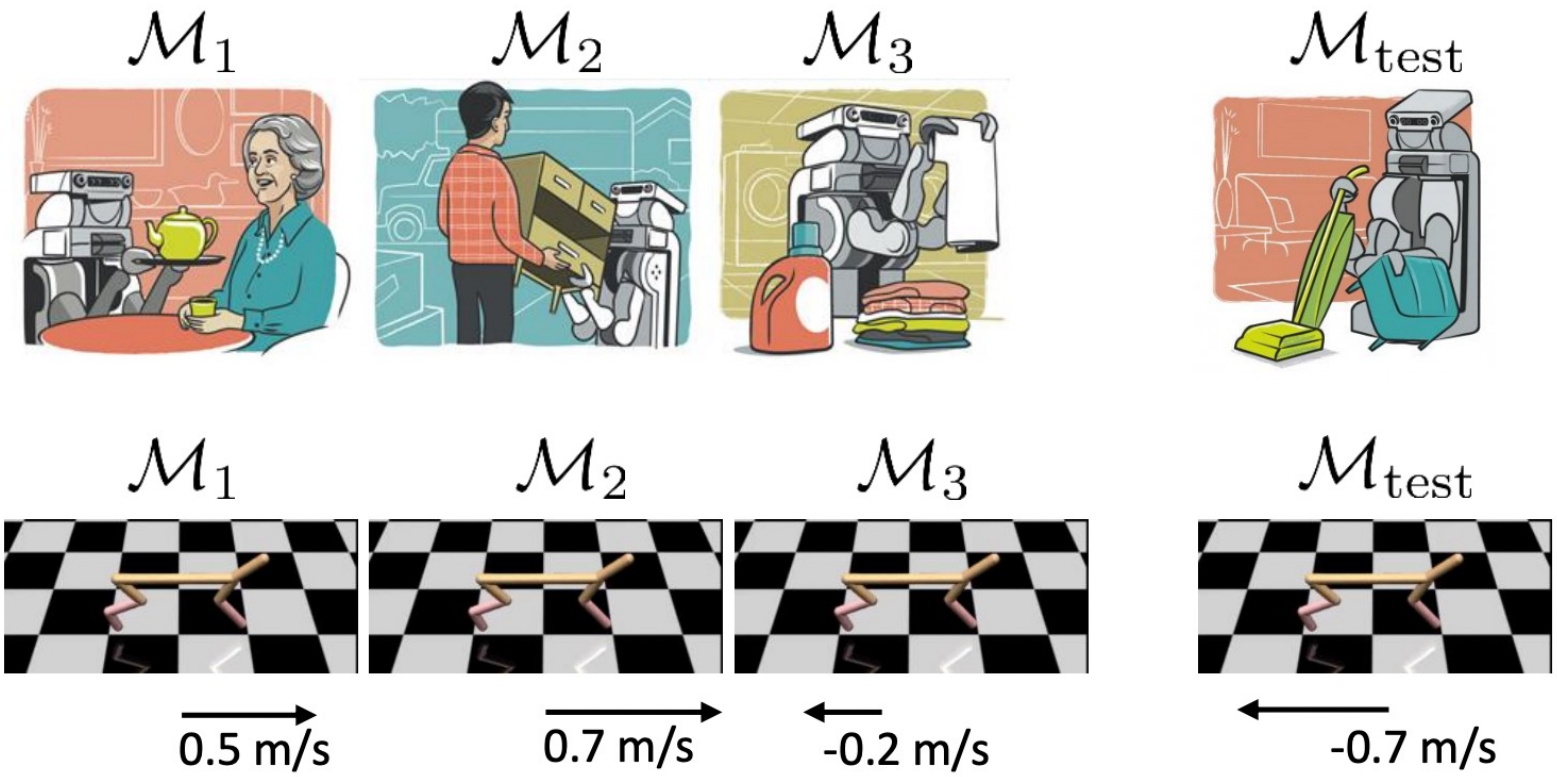
- Meta-learning with supervised learning





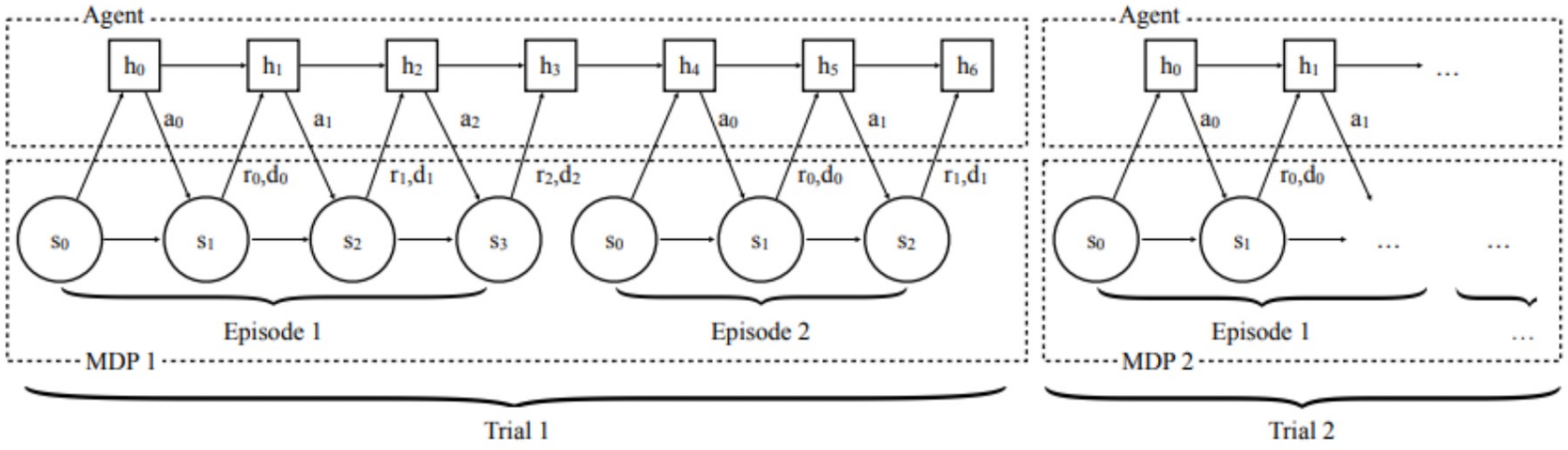
Meta Reinforcement Learning

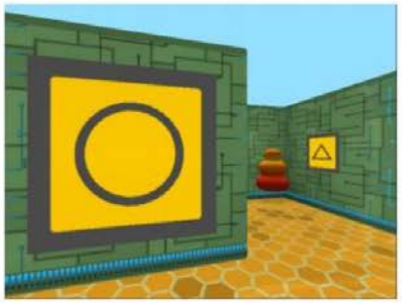
- Learn to learn new task more efficiently



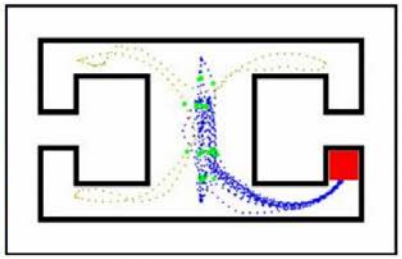
Meta Reinforcement Learning

- Meta RL with recurrent policy learning
- Similar to “In-context learning” in LLM

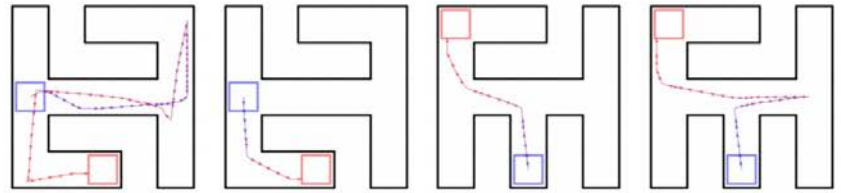




(a) Labryinth I-maze



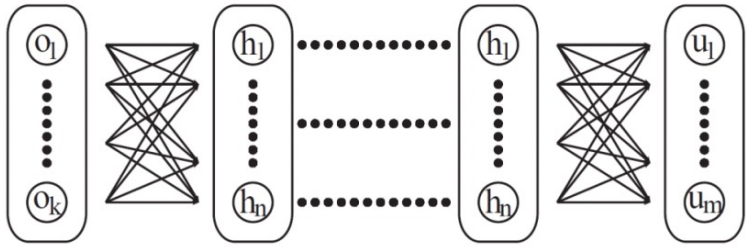
(b) Illustrative Episode

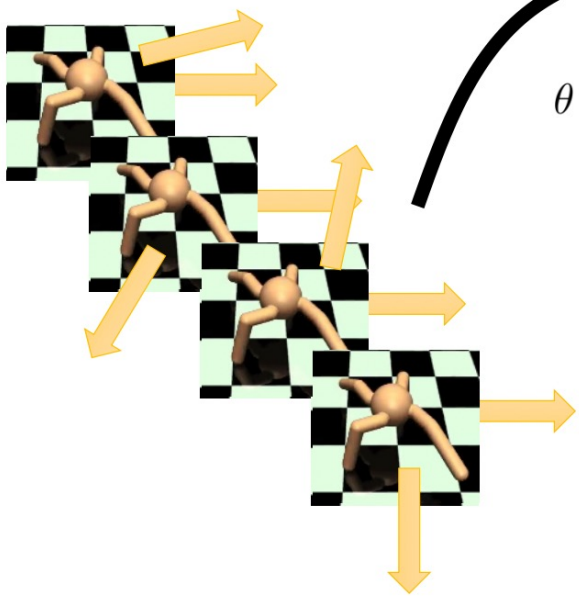


(a) Good behavior, 1st episode (b) Good behavior, 2nd episode (c) Bad behavior, 1st episode (d) Bad behavior, 2nd episode

Meta Reinforcement Learning

- Gradient-based meta RL

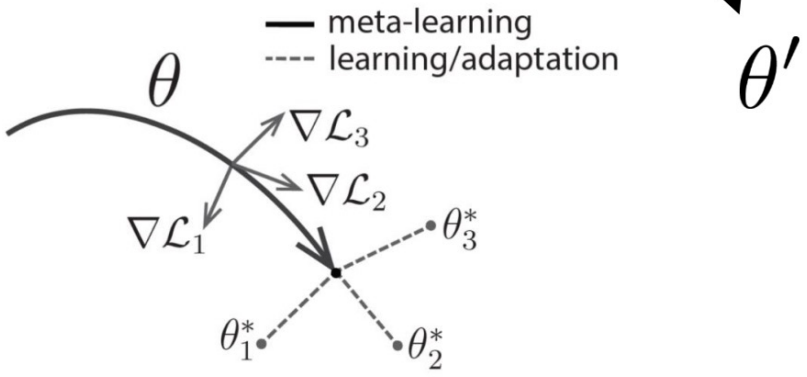




θ

~~$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$~~

$$\theta \leftarrow \theta + \beta \sum_i \nabla_\theta J_i[\theta + \alpha \nabla_\theta J_i(\theta)]$$

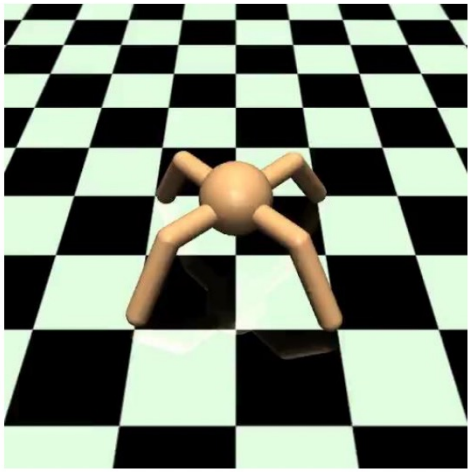


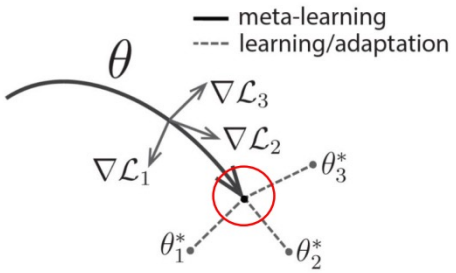
θ'

Meta Reinforcement Learning

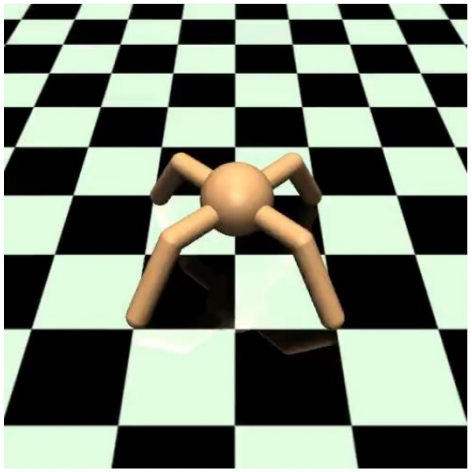
- Gradient-based meta RL

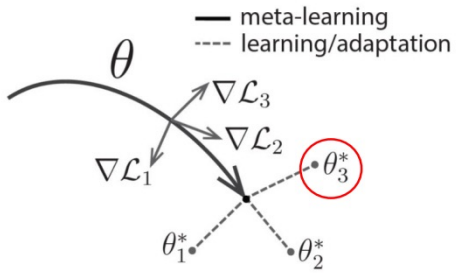
after MAML training



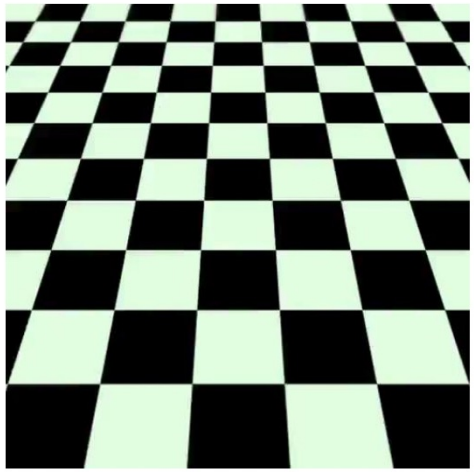


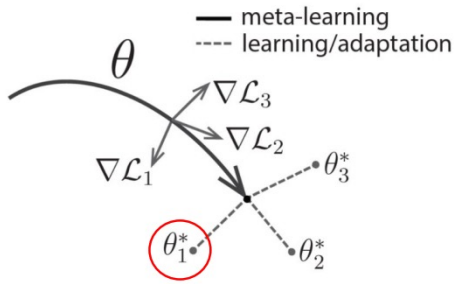
after 1 gradient step (forward reward)





after 1 gradient step (backward reward)





Thank you!

清華大學 交叉信息研究院

Institute for Interdisciplinary Information Sciences, Tsinghua University

ISR Lab

Intelligent Systems and Robotics Lab